

Compiladores I

Alexandre Marcelino Silva

(alexandre@salesiano-ata.br - alexandre@vidadura.org)

Agosto de 2006

1. TEORIA DOS COMPILADORES 2

1.1. SISTEMA DE TEMPO DE EXECUÇÃO 2
1.2. INTERPRETADORES 3
1.3. AS FASES DE UM COMPILADOR 3
- *1.3.1. Análise* *6*
- *1.3.2. Síntese* *9*

2. ANÁLISE LÉXICA 12

2.1. ANALISADOR LÉXICO 12
2.2. ATRIBUTOS PARA TOKENS 13
2.3. ERROS LÉXICOS 13
2.4. BUFFERIZAÇÃO DA ENTRADA 14
2.5. ESPECIFICAÇÃO DE TOKENS 15
2.6. STRINGS ALFABETOS E LINGUAGENS 15
2.7. OPERAÇÕES EM LINGUAGENS 15
2.8. EXPRESSÕES REGULARES 16
2.9. PROPRIEDADES ALGÉBRICAS DE EXPRESSÕES REGULARES 16
2.10. DEFINIÇÕES REGULARES 16
2.11. RECONHECIMENTO DE TOKENS 17
2.12.DIAGRAMAS DE TRANSIÇÃO 18
2.13. AUTÔMATOS FINITOS (DIAGRAMAS DE TRANSIÇÃO OU MÁQUINAS DE ESTADOS FINITOS MEF) 25
- *MEF determinista* *26*
- *MEF indeterminista* *26*
- *Equivalência entre MEFs indeterministas e MEFs deterministas (Teorema 2.1 Passagem de Ind para Det)* *28*
- *MEF indeterminista com transições espontâneas – MEFind-ε* *29*
- *Equivalência entre MEFs ind-ε e MEFind (Teorema 2.2)* *30*
- *Expressões regulares* *30*
- *Equivalência entre MEFs e expressões regulares* *31*
- *De expressões regulares para MEFind_ε (Teorema 2.3.)* *31*

3. ANÁLISE SINTÁTICA 33

3.1. TRATAMENTO DOS ERROS DE SINTAXE 34
- *Estratégias de Recuperação de Erros* *36*

3.2. GRAMÁTICAS LIVRES DE CONTEXTO 38
- *Árvores Gramaticais* *41*
- *Ambigüidade* *41*
- *Associatividade dos Operadores* *42*
- *Precedência de Operadores* *42*
- *Restrições na prática de gramáticas* *43*

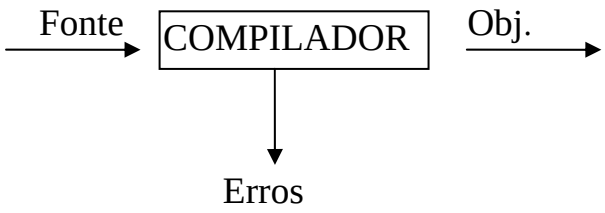
3.3. PROJETO DE ANALISADOR SINTÁTICO 43
- *Elementos Básicos e Especificações de EBNF* *44*
- *< Elementos Básicos e Especificações de Notação Formal* *46*

ESPECIFICAÇÃO DE GLC 46
3.3.1. ANÁLISE SINTÁTICA DESCENDENTE RECURSIVA 47
- *Analisadores Preditivos* *48*

3.4. ESPECIFICAÇÕES LEX 49

1. Teoria dos Compiladores

Um compilador é um programa que lê um programa escrito em uma linguagem (linguagem fonte) e o traduz em um outro programa equivalente em outra linguagem, mais próxima da máquina, (linguagem objeto). Como uma parte importante do processo de compilação, o compilador deve informar ao seu usuário a presença de erros no programa fonte.



Um programa executável é produzido pela combinação de vários arquivos objeto pelo ligador (linker). O linker agrupa o código de máquina de vários arquivos objeto, nesse agrupamento o linker tem que lidar com a realocação de endereços.
Por exemplo, para o ligador agrupar o código objeto de dois arquivos objeto, A.obj e B.obj , o linker vai adicionar aos endereços de B o tamanho de A (e.g., 800), assim se houvesse uma chamada de função em B que ocupava o endereço 100 ela passaria a ocupar o endereço 900.
Ao executar o programa.exe pode ser necessário modificar esses endereços novamente, por exemplo, se o programa executável foi carregado no endereço de memória 5000, a função que estava no endereço 900 do executável irá ocupar endereço 5900 na memória.

1.1. Sistema de Tempo de Execução

Qualquer programa compilado requer um sistema de tempo de execução (run-time system). O sistema de tempo de execução (STE) é formado por todos algoritmos de um programa executável que não são especificados diretamente pelo programador. Algumas das funções do STE são descritas a seguir.

- Alocar memória para as variáveis locais das funções antes do inicio da execução das mesmas e desalocar ao término da execução.
- Gerenciar o armazenamento do endereço de retorno da chamada de função.
- Fazer a coleta do lixo de tempos em tempos.
- Converter valores de um tipo para outro.
- Obter a linha de comando do sistema operacional.
- Etc.

Parte do código do STE é fornecido com bibliotecas já compiladas e parte é acrescentado pelo compilador. No primeiro caso, temos o caso da coleta de lixo e o código que obtém a linha de comando do sistema operacional.

1.2. Interpretadores

Um compilador traduz o código fonte para linguagem de máquina que é modificada pelo linker para produzir um programa executável.
Um interpretador traduz um programa em um pseudocódigo que ele mesmo executa. Este pseudocódigo é uma versão simplificada da linguagem de máquina.
Existem vantagens e desvantagens no uso de compiladores/linkers e m relação ao uso de interpretadores, elas estão resumidas abaixo:

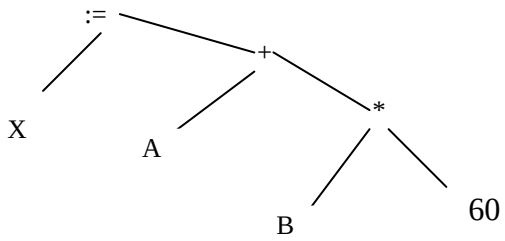
1. O código interpretado pelo interpretador nunca interfere com outros programas. O interpretador pode conferir todos os acessos à memória a ao hardware impedindo operações ilegais.
2. Com o código interpretado torna-se fácil construir um debugger pois a interpretação está sob controle do interpretador e não do hardware.
3. Interpretadores são mais fáceis de construir, pois, o pseudocódigo é mais simples que a linguagem de máquina. Eles não necessitam de linkers (a ligação é dinâmica) e o STE está presente no próprio interpretador, já o STE de um programa compilado deve ser, pelo menos em parte, codificado em assembler.
4. O tempo total para realizar todas as operações em um ambiente que emprega compilação é substancialmente maior do que se a interpretação fosse usada.
5. Compiladores produzem código mais eficiente, pois o código produzido pela compilação será executado pela própria máquina.

Concluímos que os interpretadores são melhores durante a fase de desenvolvimento de um programa, quando o programa estiver pronto é importante que ele seja o mais rápido possível, o que pode ser obtido compilando-o.

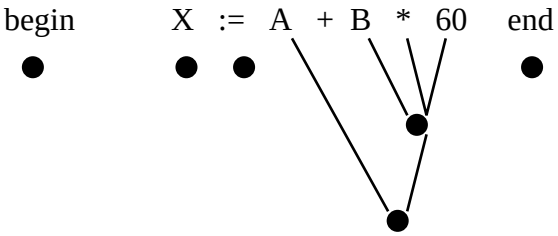
1.3. As fases de um Compilador

Podemos construir compiladores para uma grande variedade de linguagens e máquinas usando as mesmas técnicas básicas.
O processo de compilação compreende duas fases: fase de análise e fase de síntese. A fase de análise "quebra" o programa fonte em suas partes constituintes e cria uma representação intermediária do mesmo. Nessa fase o compilador percorre o programa fonte coletando seus elementos constituintes e armazenando esta informação em uma estrutura hierárquica chamada árvore sintática. Na fase de síntese o compilador usa esta representação intermediária gerada durante o processo de análise para traduzir o programa fonte para código objeto. É nessa última fase que podem ocorrer otimizações no código gerado. Para a obtenção de um programa objeto executável é necessário outros programas que trabalham junto com o compilador, tais como ligador, carregador, etc.
Na árvore sintática, cada nó representa uma operação e o ramo filho do nó representa os argumentos da operação.

Exemplo: X := A + B * 60

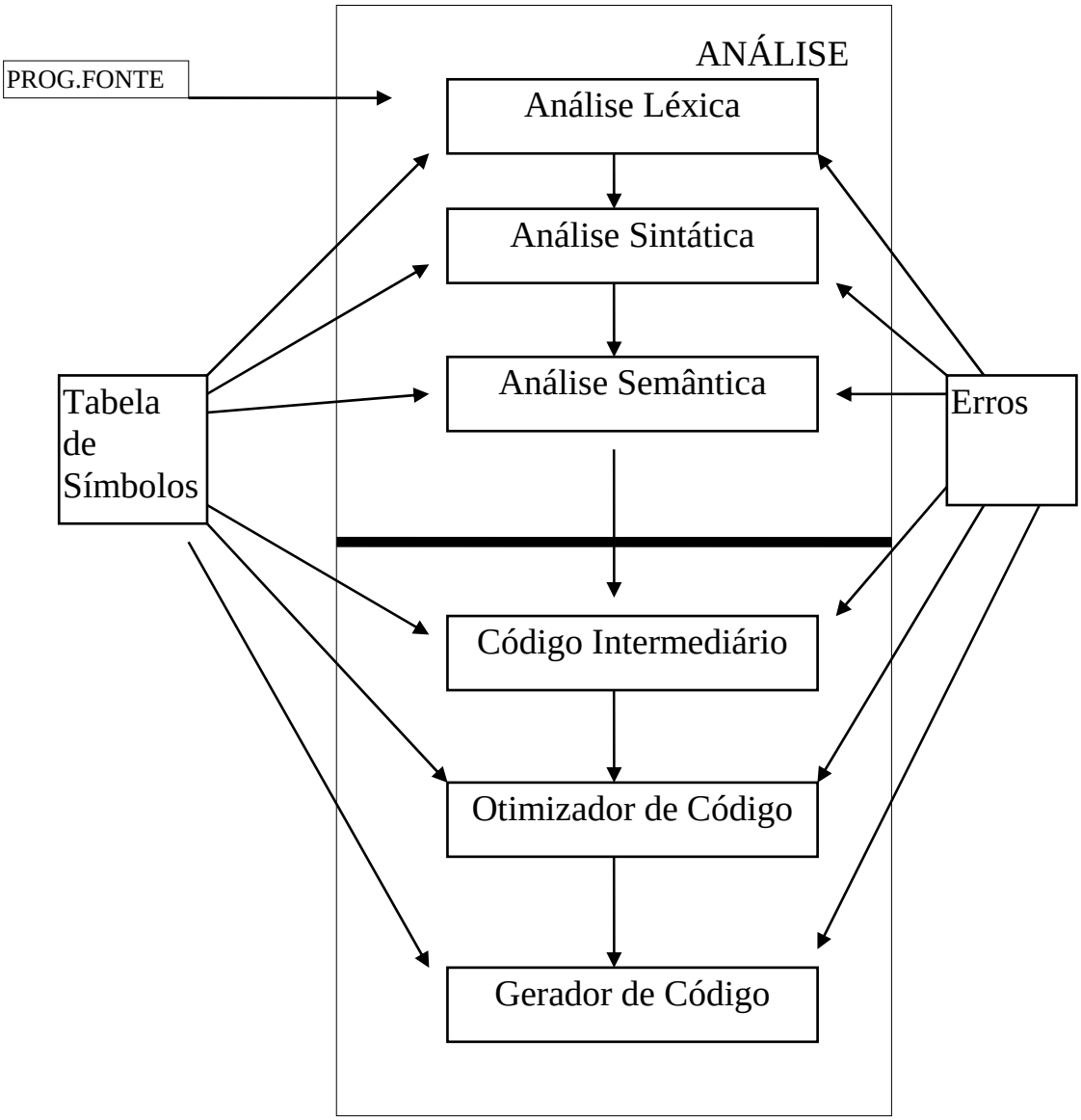


No programa-fonte podem ser encontradas unidades sintáticas, que especificam a estrutura gramatical do programa.
Ex:



Pode-se notar que há unidades sintáticas compostas por outras num segundo nível e unidades elementares no primeiro nível, estas unidades elementares são chamadas de itens léxicos.

ARQUITETURA CLÁSSICA DE UM COMPILADOR



- **1.3.1. Análise**

A análise consiste de 3 fases:

1. Análise linear ou léxica -> os caracteres do programa-fonte são lidos e agrupados em tokens.
2. Análise hierárquica ou sintática -> os tokens são agrupados em uma estrutura hierárquica com significado coletivo- árvore sintática.
3. Análise semântica -> certas verificações são realizadas para assegurar que os componentes do programa se combinam de forma significativa.

Análise Léxica

A análise linear é chamada de análise léxica ou scanning. O analisador léxico lê o programa-fonte e reconhece dentro deste cada item léxico. Na análise léxica, o programa fonte é percorrido e o analisador extrai os elementos básicos da linguagem chamados tokens. Os tokens são as palavras que formam a linguagem. Por exemplo, no Pascal, alguns tokens seriam PROGRAM, BEGIN, END, :=, identificadores, constantes, etc. O analisador léxico é chamado por demanda pelo analisador sintático, como veremos mais adiante. A cada vez que o léxico for chamado, devolve um token válido da linguagem. Ocasionalmente, quando detectar um símbolo não reconhecido, deve ser capaz de retornar o erro apropriado.

Exemplo 1: X := A + B * 60, seria agrupado nos seguintes tokens

1. identificador X
2. atribuição :=
3. identificador A
4. sinal +
5. identificador B
6. multiplicador *
7. digito 60

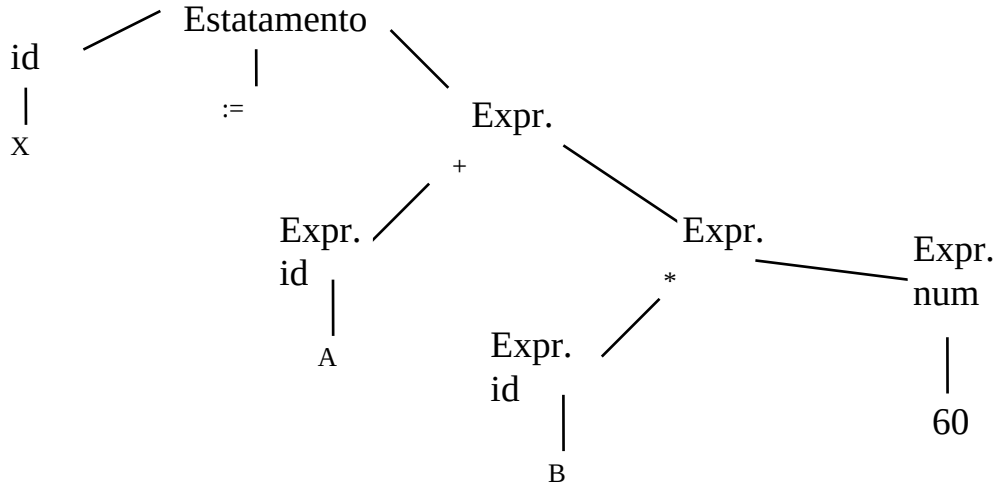
Os brancos separando os caracteres destes tokens são eliminados durante a análise léxica. O analisador léxico trabalha com os caracteres do programa-fonte, agrupando-os no item léxico correspondente, assim ele deve reconhecer palavras reservadas, identificadores, etc.

Análise sintática (Análise hierárquica, parser ou análise gramatical)

Os itens léxicos agrupados pela análise léxica são fornecidos ao analisador sintático que os agrupa nas diversas unidades sintáticas, construindo a árvore sintática, para isso ele utiliza uma série de regras, que constituem a gramática da linguagem-fonte.

Tem por tarefa reconhecer os “erros sintáticos”, construções do programa-fonte que não estão de acordo com as regras de formação de unidades sintáticas estabelecidas na gramática. Podemos representá-la através de uma árvore-parser.
O analisador hierárquico ou sintático é comumente chamado de parser e, portanto, o processo de análise sintática é chamado de parsing. Na análise sintática, os tokens são agrupados em uma estrutura de árvore chamada árvore sintática que expressa a ordem em que esses elementos devem ser avaliados, e essa árvore deve expressar a precedência (ordem em que operações devem ser feitas, por exemplo, '*' antes de '+') e a associatividade (ordem em que operações devem ser feitas, considerando a mesma precedência) dos operadores de uma expressão da linguagem. Se um token não pode ser encaixado nessa árvore sintática, então ocorre um erro sintático, significando que o programa fonte está incorreto, pois não corresponde à definição da linguagem que o compilador deve reconhecer. Portanto, deve-se entender o analisador sintático como um reconhecedor de linguagens.

Exemplo 2:



Na expressão A + B * 60, a frase B*60 é uma unidade lógica (* vem antes da adição, de acordo com as convenções aritméticas). A estrutura hierárquica de um programa é usualmente expressa por recursividade.
As gramáticas livres de contexto são a formalização da recursividade que pode ser usada para guiar a análise sintática. A árvore parser descreve a estrutura sintática da entrada, a representação interna mais comum é dada pela árvore sintática.

Análise semântica

A fase da análise semântica verifica os erros semânticos no programa-fonte e captura informações de tipo para a fase seguinte (geração de código),checa o programa-fonte e identifica operadores e operandos de expressões e estatamentos. Um importante componente da análise sintática é a checagem de tipos. O compilador checa se cada operador recebe operandos que são permitidos pela especificação da linguagem-fonte.

Exemplo 3: A := 5

1. A foi declarado
2. Onde a foi declarado (local, global)
3. Qual o tipo de A
4. O tipo de A é compatível com o lado direito da atribuição
5. qual o endereço de A no código-objeto

Exemplo 4: Go to 5, onde 5 não foi declarado como rótulo, a sintaxe é perfeita, porém há um erro semântico.

```
....var A : integer;
        B : boolean;...

...     A + B;
```

O trecho acima está sintaticamente correto, porém com uma mistura de tipos não admissível pela linguagem.

Gerenciador da Tabela de Símbolos

Uma função essencial de um compilador é registrar os identificadores usados no programa-fonte e coletar informações sobre vários atributos para cada identificador. Estes atributos provêem informações sobre a memória reservada para o identificador, seu tipo, escopo e no caso de nome de procedimentos, número e tipo de seus argumentos, o método de passagem de parâmetros e o tipo retornado, se houver.
Uma **Tabela de Símbolos** é uma estrutura de dados contendo um registro para cada identificador com campos contendo os atributos de seus identificadores.
Quando um identificador é detectado pelo analisador léxico, é colocado na Tabela de Símbolos, entretanto os atributos não são identificados na análise léxica, as fases seguintes identificam e colocam os atributos na tabela e então usam esta informação de vários modos. Por exemplo, ao realizar a análise semântica e geração de código intermediário, precisamos conhecer de que tipos os identificadores são, a fim de verificar se o programa-fonte os usa de forma válida.

Detecção de Erros e Geração de relatórios

Cada fase pode encontrar erros. Entretanto, após encontrá-los, precisa lidar de alguma forma com os mesmos, de tal forma que a compilação possa continuar, permitindo que sejam detectados erros posteriores no programa-fonte. Um compilador que pare ao encontrar o primeiro erro é “furado”.
As fases de análise sintática e semântica tratam usualmente de uma ampla fatia dos erros detectáveis pelo compilador. A fase de análise léxica pode detectá-los quando os caracteres entrados não formam nenhum token da linguagem, os erros onde o fluxo de tokens viole as regras estruturais (sintaxe) da linguagem, são determinados pela análise

sintática. Durante a análise semântica, o compilador tenta detectar as construções que possuam a estrutura sintática correta.

- **1.3.2. Síntese**

Geração de código intermediário

Depois da análise sintática e semântica, alguns compiladores geram uma representação intermediária explicita do programa-fonte. Esta representação pode ter uma variedade de formas como, por exemplo, a chamada "Tree-adress code", que é como uma linguagem de montagem para uma máquina, na qual cada localização de memória possa atuar como um registrador. O "Código de três endereços" consiste em uma seqüência de instruções, cada uma delas possuindo no máximo três operandos.
Exemplo 5:
X := A + B * 60

temp1 := inttoreal(60)
temp2 := id_3 * temp1
temp3 := id_2 + temp2
id_1 := temp3

Esta forma intermediária possui várias propriedades. Primeiro, cada instrução de três endereços, possui, no máximo, um operador, além dos de atribuição. Então, ao gerar estas instruções, o compilador precisa decidir sobre a ordem em que as mesmas devam ser realizadas. Segundo, o compilador precisa gerar um nome temporário para receber o valor computado em cada instrução. Terceiro, algumas instruções de três endereços possuem menos do que três operandos.

Otimização de Código

A fase de otimização tenta melhorar o código intermediário, de tal forma que venha resultar um código de máquina mais rápido em tempo de execução. Algumas otimizações são cruciais
Há muitas maneiras de se otimizar o código.
Exemplo 6:

temp1 := id_3 * 60.0
id_1 := id_2 + temp1

Geração de Código

A fase final do compilador é a geração do código-objeto, consistindo normalmente de código assembly. As localizações de memória são selecionadas para cada uma das variáveis usadas pelo programa. Então, as instruções intermediárias são, cada uma, traduzidas numa seqüência de instruções de máquina que realizam a mesma tarefa.

Exemplo 7:

MOV id_3 , R1
MUL #60.0, R2
MOV id_2 , R1
ADD R2, R1
MOV R1, id_1

OS PRIMOS DO COMPILADOR

Pré-processadores

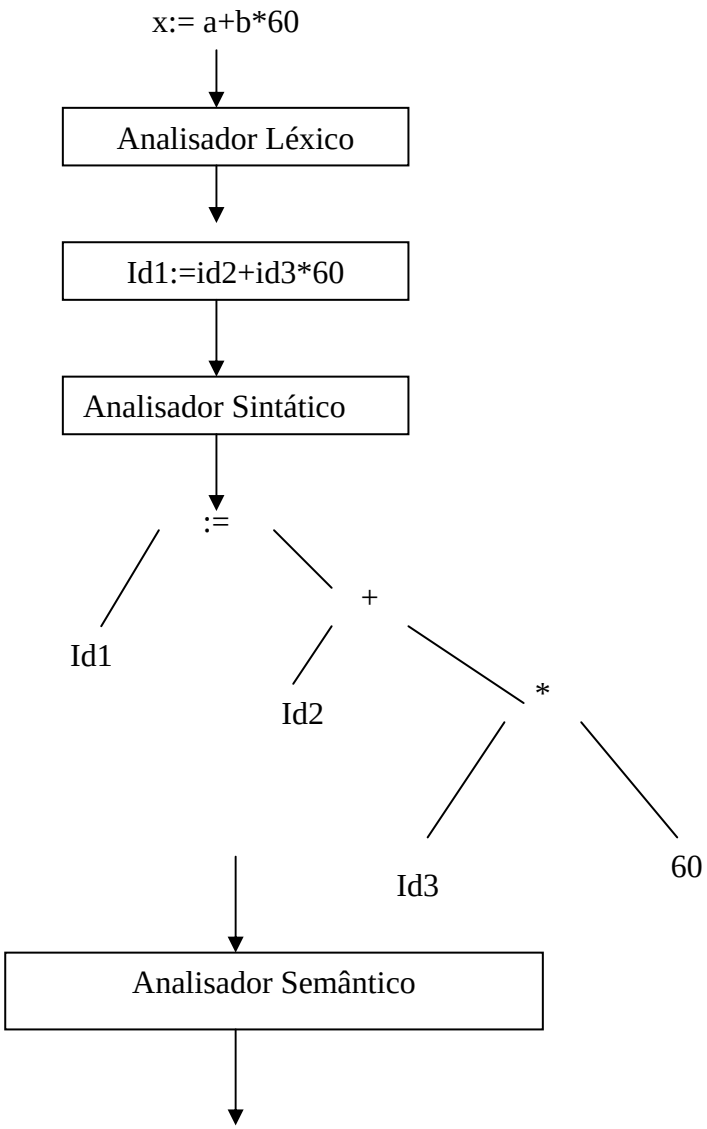
Eles produzem entradas para os compiladores. Realizam as seguintes funções:

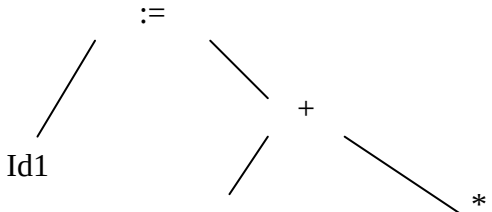
1. Processamento de macros
2. Inclusão de arquivos (headers,e.g.)
3. Pré-processadores racionais - incrementam controles de fluxo e facilidades de estruturação á linguagens antigas
4. Extensão de linguagem - adicionam, através de macros, capacidades adicionais a linguagem

Montadores

Alguns compiladores produzem um código de montagem que é passado a um montador para processamento posterior. Outros compiladores realizam a tarefa do montador, produzindo código de máquina relocável, que pode ser passado diretamente para um carregador/editor de ligações.

Exemplo 8: A figura abaixo ilustra a tradução de uma declaração em todas fases de compilação.


x:= a+b*60
Analisador Léxico
Id1:=id2+id3*60
Analisador Sintático
:=
Id1
+
Id2
*
Id3
60
Analisador Semântico

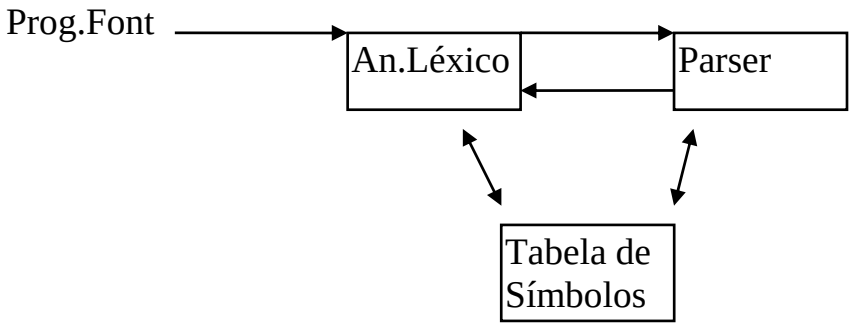

:=
Id1
+
*

2. Análise Léxica

Uma maneira simples de implementar analisadores léxicos é construir um diagrama que ilustre a estrutura dos tokens da linguagem-fonte e então converter o diagrama em um programa para encontrar tokens. Esta mesma técnica pode ser aplicada em outras áreas tais como: linguagens query e sistemas de recuperação de informações.
O problema básico é a especificação e design de programas que executem ações disparadas por padrões nas cadeias de caracteres. Portanto, usaremos a linguagem LEX para especificar o analisador léxico (para quem optar pelo projeto de linguagem). O LEX pode gerar um eficiente autômato finito reconhecedor para expressões regulares (padrões são expressões regulares).

2.1. Analisador Léxico

É a primeira fase de um compilador, sua principal tarefa é ler caracteres de entrada e produzir como saída uma seqüência de tokens que o parser usa para análise sintática. Esta interação é implementada fazendo o analisador léxico ser uma co-rotina do parser. Depois de receber um comando "busque o próximo token" do parser, o analisador léxico lê os caracteres da entrada até identificar o próximo token.



Outra tarefa do analisador léxico é retirar os comentários e espaços em branco como brancos, tabulações, newlines e correlacionar mensagens de erro.
Quando falamos sobre análise léxica, nós usamos os temos "Token", "Lexema" com significado específico.

Token	Lexema	Designação Do Padrão
const	const	const
id	pi, cont, O2	letra seguida por letra e dígitos
literal	"qualquer"	qq. caracter entre "e", exceto a própria "

Em geral, há um conjunto de strings na entrada para qual o mesmo token é produzido como saída, este conjunto é descrito pelo padrão associado com o token.
Nós tratamos tokens como símbolos terminais na gramática da linguagem-fonte na maioria das linguagens de programação as seguintes construções são tratadas como tokens: palavras-chave, operadores, identificadores, constantes, literais e símbolos de pontuação.
Por exemplo, quando o identificador pi aparece no programa-fonte, um token representando um identificador é retornado para o parser.
O retorno de um token é geralmente implementado pela passagem de um inteiro correspondendo ao token.
O padrão descreve o conjunto de lexemas que pode representar um token. Assim, o padrão para o token Relação é o conjunto de operadores relacionais, para descrever padrões mais complexos como id,por exemplo, usa-se a notação de expressão regulares.
Certas convenções de linguagem aumentam a dificuldade da análise léxica, linguagens como Fortran requerem certas construções em posições fixas na linha de entrada. Dessa forma, o alinhamento de um lexema pode ser importante na determinação de correção de um programa-fonte.

2.2. Atributos para Tokens

Quando um lexema for reconhecido por mais de um padrão, o analisador léxico deve providenciar informações adicionais sobre o lexema para as fases subseqüentes do compilador.
Por exemplo, o padrão **num** reconhece as duas cadeias 0 e 1, mas é essencial para o gerador de código ser informado sobre que cadeia foi efetivamente reconhecida.
Na prática, um token possui apenas um atributo - um ponteiro para a entrada na Tabela de símbolos na qual a informação sobre ele está armazenada.

2.3. Erros Léxicos

Poucos erros são discerníveis nesta fase, porque o analisador léxico tem uma visão muito local do programa-fonte.
Se a string **fi** for encontrada no contexto de um programa C,

```
fi (a==f (x))...
```

o analisador léxico não pode dizer se fi é uma forma errada da palavra-chave if ou um identificador de função não declarado. Desde que fi é válido como identificador, o analisador léxico deve retornar o token id para o parser e outra fase da compilação tratará o erro.
Possíveis ações de recuperação de erros são:
1. remover um caracter estranho
2. inserir um caracter ausente
3. substituir um caractere incorreto por um correto
4. transpor dois caracteres adjacentes

Transformações de erros como essas podem ser experimentadas numa tentativa de se consertar a entrada. A mais simples de tais estratégias é a de verificar se um prefixo de entrada remanescente pode se transformado num lexema válido através de uma única transformação. Essa estratégia assume que a maioria dos erros léxicos seja resultante de um único erro de transformação, uma suposição usualmente confirmada na prática, embora nem sempre.

2.4. Bufferização da Entrada

Há 3 métodos para implementar-se um analisador léxico.

1. Geradores automáticos (e.g.,LEX) - o gerador provê rotinas para ler e bufferizar a entrada.
2. Escrever usando uma linguagem de programação de alto nível, usando as facilidades de I/O da linguagem.
3. Escrever em Assembly e explicitar o gerenciamento da leitura.

Para muitas linguagens, o analisador léxico necessita ler muitos caracteres do lexema até que seja anunciado um reconhecimento, técnicas de bufferização devem ser usados para minimizar a sobrecarga requerida para processar a entrada.
Pode-se usar, por exemplo, um buffer dividido em duas partes de N - caracteres (N é o número de caracteres de um bloco do disco, 1024 ou 4096).
Nos lemos N caracteres de entrada para cada lado do buffer com um comando de leitura, ao invés de invocar um comando de leitura para cada caracter de entrada.
Se restarem menos do que N caracteres na entrada, então um caracter especial eof é lido para o buffer depois dos caracteres de entrada, **eof** marca o final do arquivo-fonte e é diferente de qualquer caracter de entrada
São mantidos dois ponteiros para o buffer de entrada. A string de caracteres entre os dois ponteiros é o lexema corrente, inicialmente, ambos os apontadores indicam o primeiro caracter do próximo lexema a ser levantado
O ponteiro que chamaremos de forward rastreia até que encontra uma combinação para um padrão.

Exemplo 9:
```
if forward at end of first half then begin
    reload second half,
    forward:  = forward + 1
end
else if forward at end of second half then begin
     reload first half,
end

else forward : = forward +1;

Código para avançar o ponteiro...
```

2.5. Especificação de Tokens

Expressão regulares são uma importante notação para especificar padrões. Cada padrão corresponde a um conjunto de strings, então expressões regulares também servem como nomes para conjunto de strings.

2.6. Strings Alfabetos e Linguagens

Um *símbolo* é uma entidade abstrata, letras e dígitos são exemplos de símbolos. O termo *Alfabeto* denota qualquer conjunto finito de símbolos (e.g., letras e caracteres). O conjunto {0, 1} é o alfabeto binário.
Uma *string* (seqüência) de algum alfabeto é uma seqüência finita de símbolos justapostos daquele alfabeto. O termo *Linguagem* denota qualquer conjunto de strings de um alfabeto fixo.

2.7. Operações em Linguagens

Há importantes operações que podem ser aplicadas em linguagens. Para a análise léxica nos interessa união, concatenação e fecho.

Exemplo 10:

L = { A,B, ...,Z, a,b,....z}
D = {0,1,....9}

Podemos dizer que "L" é o alfabeto consistido de letras maiúscula e minúsculas e "D" é o alfabeto consistido dos números decimais.
Podemos criar novas linguagens de "L" e "D", aplicando os operadores.

1. L U D : Conjunto de letras e dígitos
2. L D : Uma letra seguida por um dígito
3. L^4 : Conjunto de todas strings de 4 letras
4. L^*: Todas strings de letras, incluindo E
5. $L (L U D)^*$: Todas strings de letras e dígitos começando com uma letra
6. D^+ : Todas strings de um ou mais dígitos

Operação	Definição
União de L e M $\rightarrow$ L U M	L U M = [S\S is in L or S is in M}
Concatenação LM	LM = { St\S is in L and t is in M}
Fecho L^*	$L^* = U^{\infty}_{i=0} L^i$ (zero ou + concatenações de L)
Fecho positivo L^+	$L^+ = U^{\infty}_{i-1} L^i$ (1 ou +

2.8. Expressões Regulares

Uma expressão regular é construída de expressões regulares simples usando um conjunto de regras de definição. Cada expressão regular r denota uma linguagem L(r) . L(r) é formada pela combinação de vários sub-expressões de r.

Exemplo 11: Em pascal, um identificador

Letter (letter \ digit)*

Uma linguagem denotada por uma expressão regular é dita um conjunto regular.

Exemplo 12:

1. a | b* c → Denota o conjunto de strings que são ou um simples a ou zero ou mais b's seguidos por um c.
2. a | b → Denota o conjunto { a,b}
3. (a | b) (a | b) → Denota { aa, ab, ba, bb}
4. a* → Zero ou + a's → { E, a, aa, aaa....}
5. (a | b)* → Zero ou + instâncias de a ou b
6. a | a* b → a ou zero ou + A's seguido por um b.

Se r e s denotam a mesma linguagem, nós dizemos que r e s são equivalentes e escrevemos r + s, por exemplo. (a | b) = (b | a)

2.9. Propriedades algébricas de expressões regulares

r | s = s | r	| é comutativa
R | (s | t) = (r | s) | t =	| é associativa
(r s) t = r (s t)	concatenação é associativa
r (s | t) = r s | r t (s | t) r = s r | r t	concatenação distribuída sobre
E r = r r E = r	E é o elemento identidade para concatenação
r* = (r | E)*	relação entre fecho e E
r** = r*	* é indepotente

2.10. Definições Regulares

Por conveniência de notação, nós podemos desejar dar nomes a expressões regulares, bem como definir outras, usando estes nomes como se fossem símbolos.
Se ∑ é um alfabeto de símbolos básicos, então uma definição regular é uma seqüência da forma:

d1 → r1
d2 → r2 . . .
.
Onde cada d i é um nome distinto e cada r i é uma expressão regular.

Exemplo 13:

Letter → A|B|....|Z|a|b....\z
digit → 0|1....|9
id → letter (letter | digit)*

Certas construções ocorrem tão freqüentemente em expressões regulares que é conveniente introduzir notações simplificadas para elas.

1. Uma ou + instâncias (+)→ $r^+ = r\,r^*$
2. Zero ou uma instância → (?) significada 0 ou 1 instância

Exemplo 14:

digit → 0 |1|.....| 9
digits → digit +
fractions → (. Digits)?
Exponent → (E (+ \ -) ? digits) ?
num → digits fraction exponent

3. Classe de Caracteres → uma classe de caracter como (a - z) denota a expressão regular

a | b | ...| z
i d → [A - Z a -z] [A - Z a - z 0 -9] *

Algumas linguagens não podem ser descritas por qualquer expressão regular.
Strings não podem ser descritas por expressões regulares simples.
Exemplo:
{ w c w | w é uma string de a's ou b's } não pode ser descrita por uma expressão regular, mas pode ser descrita por uma gramática livre de contexto.

2.11. Reconhecimento de Tokens

Consideremos a seguinte gramática:

s t m t → if **expr** then **stmt**
| if **expr** then **stmt** else **stmt**
| E
expr → **term** relop **term**

| **term**

term → id
| num

Onde os terminais if, then, else, relop, e num gera conjuntos de strings dados pela seguinte definição regular:

if → if
then → then
else → else
relop → < \ <= \ = \ <> \ > \ >=
id → letter (letter | digit) *
num → digit + (.digit ?)? (E + \ -) ? digit ?) ?

Onde letter e digit foram previamente definidos.
Para este fragmento de linguagem o analisador léxico irá reconhecer as palavras-chaves if, then, else, bem como os lexemas denotados por relop, id e num.
Assumimos que lexemas são separados por espaços em branco, consistindo de brancos e tabs e newlines. Para retirar os espaços em branco, ele irá comparar uma string com a definição regular ws.

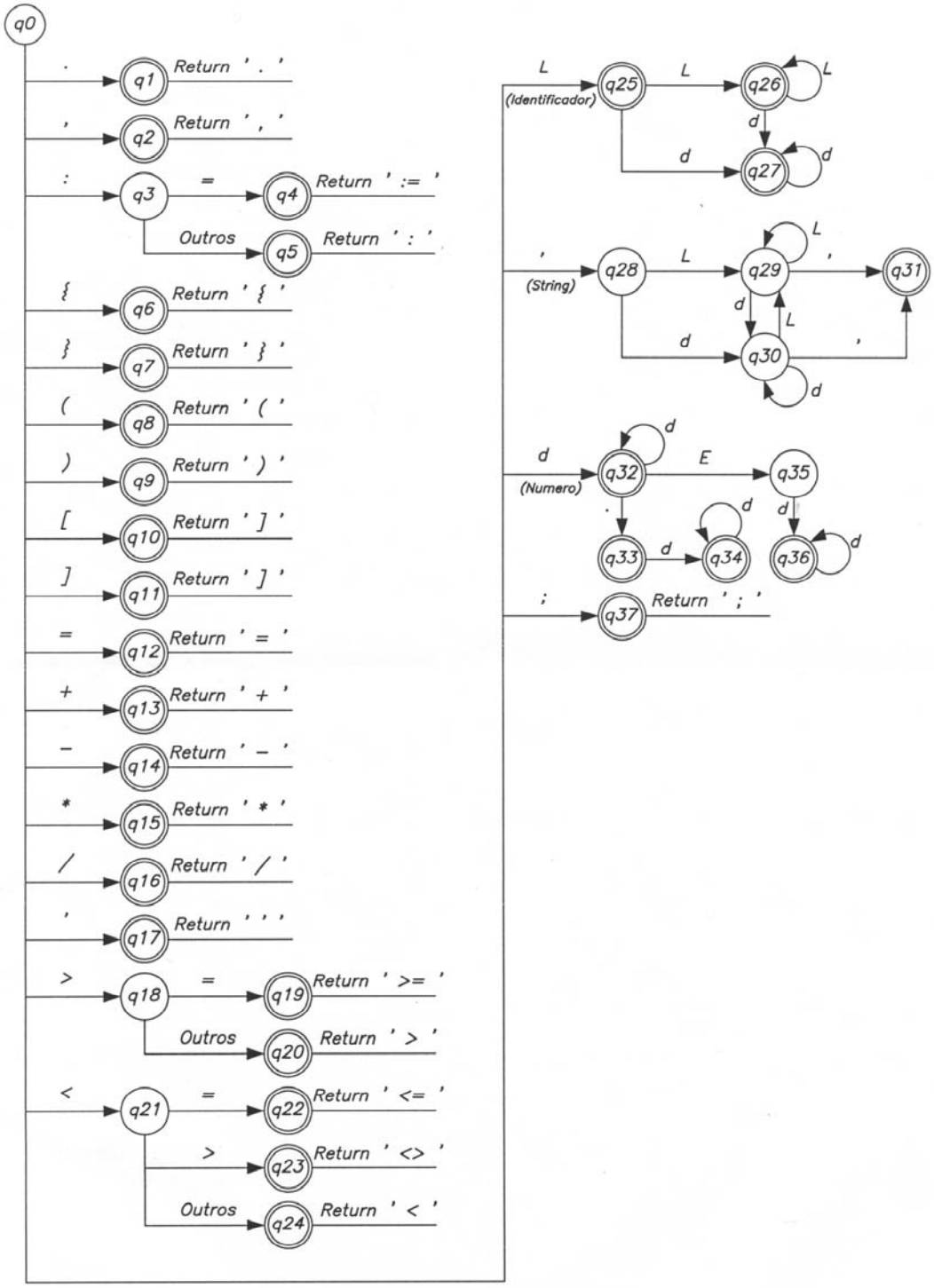
delim → blank \ tab \ newline
WS → $delim^+$

Se a combinação para ws é encontrada o analisador léxico não retorna um token para o parser, ele procede para encontrar o token seguinte ao espaço em branco.

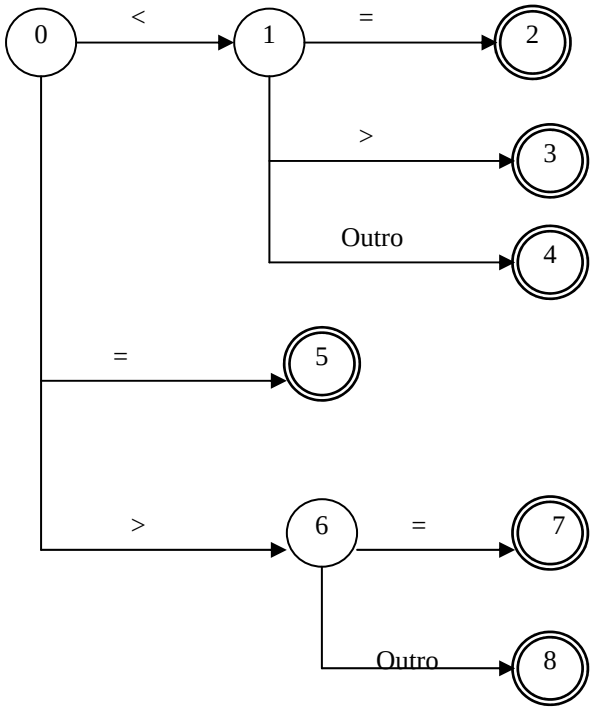
2.12.Diagramas de Transição

Uma das técnicas de reconhecimento de tokens é o uso de diagramas de transição que são a representação de um autômato finito, os quais estudaremos adiante para implementar um analisador léxico. O diagrama de transição descreve as ações realizadas quando o analisador léxico é chamado pelo parser afim de obter o próximo token
Um diagrama de transição é um grafo orientado em que os nós representam estados do autômato finito. No diagrama, os estados são representados por círculos, e o estado final por um círculo duplo. Os caracteres lidos são representados sobre os arcos que identificam as transições do autômato finito.
Sendo assim, apresentamos abaixo um diagrama para reconhecimento dos tokens válidos para um mini Pascal.
Exemplo 15: Diagrama de transições

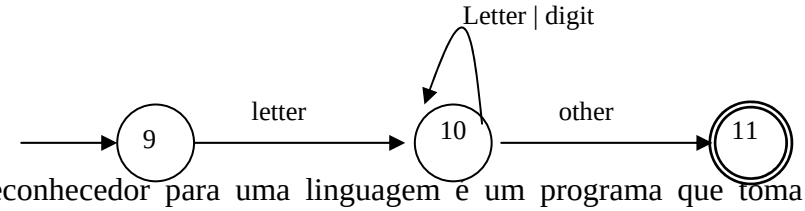
AUTOMÁTO P/ RECONHECEDOR LÊXICO



Na figura acima, onde aparecer a palavra *"**outros"*** na transição para um estado significa que o último caracter lido será devolvido para a entrada, ou seja, o ponteiro do arquivo retornará um caracter. Isso porque o último caracter pode ser o primeiro do próximo token. Cada estado final (com círculo duplo) retorna um token ao sintático.
A medida que cada token for reconhecido, poderá ser colocado numa tabela apropriada, como é o caso de identificadores, números e strings. O léxico consegue também detectar erros, do tipo 'símbolo não reconhecido pela linguagem', ou 'variável duplicada na declaração' (este último particularmente para a análise sintática).
Os diagramas de transições delineiam as ações que tomam lugar quando um analisador léxico é chamado pelo parser a fim de obter o próximo token.
Usamos um diagrama de transições para controlar as informações a respeito dos caracteres que são examinados na medida em que o apontador esquadrinhe a entrada.
Realizamos isso pela movimentação posição a posição no diagrama à medida que os caracteres são lidos. As posições são desenhadas como círculos e são chamadas de estados. Os estados são conectados por setas representando as transições, essas setas possuem rótulos indicando os caracteres de entrada. No estado inicial reside o controle quando iniciamos o reconhecimento de um token. Certos estados podem ter ações que são executadas quando são atingidos pelo fluxo de controle.
Em geral, podemos ter muitos diagramas de transição, cada um especificando um grupo de tokens. Se uma falha ocorre em todo diagrama de transição, então um erro léxico foi detectado e é invocada uma rotina de recuperação de erro.
Exemplo 16: Diagrama de transições para os operadores relacionais



Exemplo 17: Diagrama de Transições para Identificadores e Palavras-chave



Um reconhecedor para uma linguagem é um programa que toma como entrada uma cadeia x e responde ‘sim’ se x for uma sentença da linguagem e ‘não’ em caso contrário. Compilamos expressões regulares num reconhecedor através da construção de um diagrama de transições generalizado chamado de autômato finito.

- Exemplo 18: Um Analisador Léxico simples em C

```
...
//funcao lex
int Lex(void) {
      estado=0;
      while(1) {
            //switch de estados
      switch(estado) {
            //Executa a procedure que Le caracter no arquivo fonte
            case 0: c = LerCaracter();
                  //switch de caracter
             switch(c) {
                  case EOF : return 0;
                  case ' ' : estado=0; break;
                  case '\t':estado=0; break;
                  case '\n':estado=0; linha++; break;
                  case ':': estado=1; break;
                  case '>': estado=4; break;
                  case '<': estado=7; break;
                  case '=': return igual;
                  case '+': return mais;
                  case '-': return menos;
                  case '*': return vezes;
                  case '/': return dividir;
                  case ';': return ptvirgula;
                  case ',': return virgula;
                  case '\'':estado=20; break;
                  case '(': return abreparente;
                  case ')': return fechaparente;
                  case '{': return abrechave;
                  case '}': return fechachave;
                  case '.': return ponto;
                  default : if (isalpha(c)){
                        palavra[i]=c;
                        i++;
                        estado=11;}
                         else
                        if (isdigit(c))
                             estado=27;
                        else {
                               erros();
                               return err;
                        }
            }// fim switch de caracter
```

```
 break;
case 1: c=LerCaracter();
      if (c=='=')
         return doisptigual;
      else {
         volta();
         return doispt;
      }
case 4: c=LerCaracter();
      if (c=='=')
         return maiorigual;
      else {
            volta();
      return maior;
      }
case 7: c=LerCaracter();
      if (c=='=')
         return maiorigual;
      else
      if (c=='>')
         return diferente;
      else {
         volta();
         return maior;
      };
case 11: c=LerCaracter();
       if (isalpha(c) || isdigit(c)) {
            palavra[i]=c;
            i++;
            estado=11;
            break;
       }
       else {
            volta();
            palavra[i]='\0';
            return idel + inseretbsimbolo();
       }
case 20: c=LerCaracter();
       if (c=='\'')
            return frase;
       else
            if ((c=='\n')||(c==';')) {
                  erros();
         return err;
      }
      else
            estado=20;
       break;
case 27: c=LerCaracter();
       if (isdigit(c))
      estado=27;
       else
      if (c=='.')
         estado=28;
      else {
         volta();
         return numeroint;
      }
       break;
case 28: c=LerCaracter();
```

```
                if (isdigit(c))
                estado=29;
                break;
          case 29: c=LerCaracter();
                if (isdigit(c))
               estado=29;
                else
               if (c=='E')
                  estado=30;
               else {
                  volta();
                  return numeroreal;
               }
                break;
          case 30: c=LerCaracter();
                if ((c=='+')||(c=='-'))
               estado=31;
                else
               if (isdigit(c))
                  estado=32;
               else {
                     erros();
                     return err;
               }
                break;
          case 31: c=LerCaracter();
                if (isdigit(c))
               estado=32;
                break;
          case 32: c=LerCaracter();
                if (isdigit(c))
                estado=32;
                else {
               volta();
               return numeroexp;
                }
                break;
     } //fim switch de estados
      }//fim while
}
```

Obs: isalpha(), isdigit() são funções definidas em bibliotecas do C.

Exemplo 19: Um analisador Léxico para a gramática que define expressões aritméticas

Nesta exemplo faremos um pequeno analisador léxico para a gramática abaixo:

Expr::= Expr ‘+’ Termo | Expr ‘-‘ Termo | Termo
Termo::=Termo ‘*’ Num | Termo ‘/’ Num | Num
Num::=’0’|’1’|. . .’9’

Exemplo de sentenças válidas:
5+6*7
4*3*2/6*5+1
5

A cada terminal da gramática associamos uma constante enumerada

```
Enum TpToken
        {null_smb,mais_smb,menos_smb,mult_smb,div_smb,numero_smb,eof_smb}
```

Obs: num_smb é associado a todos os digitos e eof_smb corresponde ao fim de arquivo.

A função nextToken encontra o próximo token da entrada, colocando a constante correspondente na variável token.

```
int line, num;
char *text, *nextch;
tptoken token;

void nexttoken( )
{
        while(1) {
                while(*nextch== ‘ ‘ || *nextch==’\t’|| *nextch==’\r’)
                 {
                   nextch++;
                   if (*nextch==’\n’)
                          {
                            nextch++;
                            num++; }
                         }
       switch(*nextch) {
       case ‘+’: token=mais_smb; break;
       case ‘-‘: token=menos_smb; break;
       . . . /* Idem para divisão e multiplicação*/
       case ‘0’: case ‘1’: case ‘2’: . . . case ‘9’:

       num=*nextch – ‘0’;
       token=num_smb;
       break;
      case ‘\0’: token=eof_smb; break;
      default: erro(‘Caracter não identificado’)}
}
```

A variável text aponta para uma área dinamicamente alocada contendo o texto do programa a ser analisado terminado em ‘\0’. A variável nextch inicialmente aponta para o inicio desse texto e vai caminhando por ele a medida que a função nexttoken for chamada.
Sempre que nexttoken encontrar um digito, a variável token receberá num_smb, para saber o número correspondente basta consultar a variável num. O número da linha é dado pela variável line.

Inicialmente as variáveis acima serão inicializadas como:

line=1;
nextch=text;
token=null_smb;

Assumindo que o programa foi lido para a memória para uma área apontada por text, a função nexttoken é utilizada da seguinte maneira:

```
int  Expr( )
        {
          int resultado=Termo ( );
           while (token===mais_smb || token==menos_smb) {
                        tptoken aux=token;
                        nexttoken ( ); /*pegou o token '+', '-', passa para o próximo*/
                        switch(aux) {
                           case mais_smb: resultado= resultado + Termo ( ); break;
                             case menos_smb: resultado – Termo ( ); }
           } //while
          return resultado }
```

2.13. Autômatos Finitos (Diagramas de Transição ou Máquinas de Estados Finitos MEF)

Uma MEF é um modelo matemático que descreve entradas e saídas. Isto é, uma MEF é um conjunto finito de estados e um conjunto de transições de estado para estado que ocorre nos símbolos de entrada de um alfabeto W.
Uma MEF é definida formalmente pela quíntupla {Q, Σ,δ, q0, F}, onde:

Q é um conjunto finito de estados;
Σ é um alfabeto finito de entradas;
δ é uma função de transição;
q0 é o estado inicial;
e F é o conjunto de estados finais.

Uma MEF pode ser determinista ou não determinista . Ambas são capazes de reconhecer precisamente os conjuntos regulares, ou seja, ambas podem reconhecer exatamente oque as expressões regulares podem denotar.
MEFs deterministas levam a reconhecedores mais rápidos, porém muito maior e mais complexo para projetar-se diretamente. A conversão de uma expressão regular numa MEFind é mais direta (e mais fácil).

Uma MEF é representada pelos diagramas de transição. Estudaremos as MEFs deterministas, indeterministas e algumas transformações importantes para provar a correção matemática do nosso projeto.

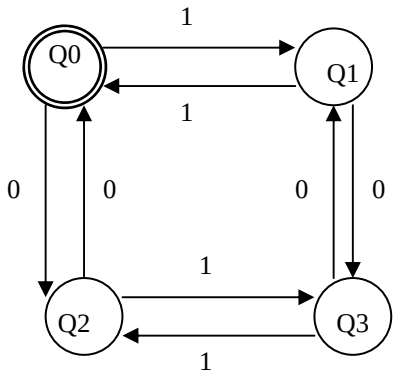
- **MEF determinista**

- Os vértices são os estados (na representação)
- δ , estando a MEF no estado q, δ (q,w) será o estado da MEF após a leitura
  - δ (q,ε) = q, ou seja, não muda de estado com ε (entrada vazia)
  - δ (q,wa) = δ(δ(q,w), a) após leitura

Exemplo 20:

Q={q0,q1,q2,q3}, q0=estado inicial, Σ = {0,1}, F={q0}

Estados	0	1
Q0	Q2	Q1
Q1	Q3	Q0
Q2	Q0	Q3
Q3	Q1	Q2



Pergunta: a seqüência 110101 é aceita pela linguagem de L(M)?
Deve-se provar que δ (q0, 110101) = q0

Realizando as transições:

δ(q0,1)=Q1
δ(q0,11)= δ(δ(q0,1), 1)= δ(q1,1) = q0
δ(q0,110)= δ(δ(q0,11),0) = δ(q0,0)= q2
δ(q0,1101)= δ(δ(q0,110),1)= δ(q2,1) = q3
δ(q0,11010)= δ(δ(q0,1101),0)= δ(q3,0) = q1
δ(q0,110101)= δ(δ(q0,11010),1)= δ(q1,1)= q0

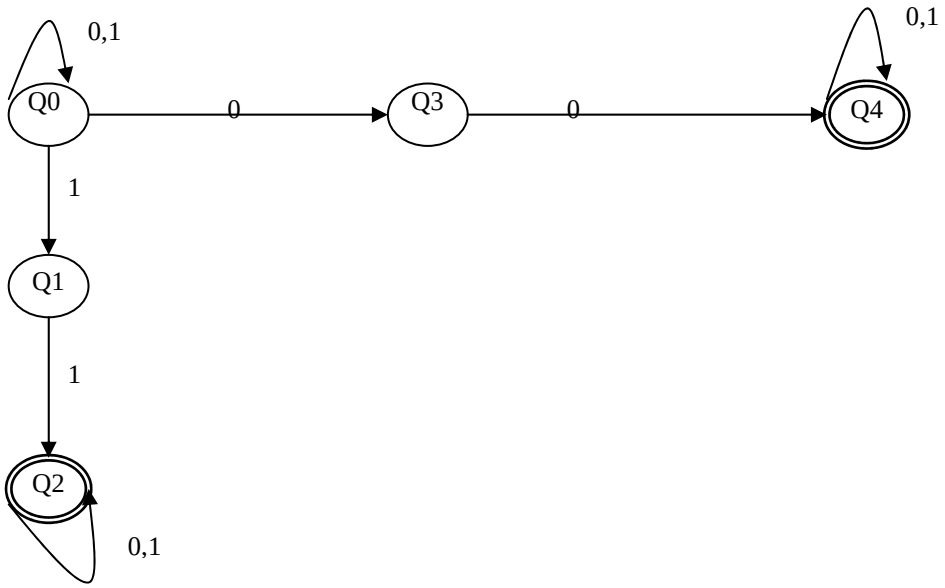
Portanto, 110101 pertence a L(M).

- **MEF indeterminista**

- é uma MEF que permite 0 ou mais transições a partir de um estado, para um mesmo símbolo de entrada
- a função de transição mapeia Q em 2^Q

Exemplo 21:

Q={q0,q1,q2,q3,q4}
Σ={0,1}
F={q2,q4}



Para descrever um sistema de forma abstrata usa-se o indeterminismo.
Na MEF acima, a partir do estado q0 há duas transições possíveis (para 0 ou 1).

- **Estendendo a função de transição**

1. $\delta(q,\varepsilon) = q$
2. $\delta(q,wa) = \{ p \mid \exists r \in \delta(q,w) \text{ e } p \in \delta(r,a)\}$
3. $\delta(P,w) = U\ q \in P\ \delta(q,w)$

Exemplo 3:

Estados	0	1
Q0	{q0,q3}	{q0,q1}
Q1	∅	{q2}
Q2	{q2}	{q2}
Q3	{q4}	∅
Q4	{q4}	{q4}

A seqüência 01001 pertence a L(M)?

Realizando as transições:

$\delta(q0,0) = \{q0,q3\}$
$\delta(q0,01) = \delta(\delta(q0,0),1) = \delta(\{q0,q3\},1) = \delta(q0,1) \cup \delta(q3,1) = \{q0,q1\} \cup \varnothing = \{q0,q1\}$
$\delta(qo,010) = \delta(\delta(q0,01),0) = \delta(\{q0,q1\},0) = \delta(q0,0) \cup \delta(q1,0) = \{q0,q3\} \cup \varnothing = \{q0,q3\}$
$\delta(q0,0100) = \delta(\delta(q0,010),0) = \delta(\{q0,q3\},0) = \delta(q0,0) \cup \delta(q3,0) = \{q0,q3\} \cup \{q4\} = \{q0,q3,q4\}$
$\delta(q0,01001) = \delta(\delta(q0,0100),1) = \delta(\{q0,q3,q4\},1) = \delta(q0,1) \cup \delta(q3,1) \cup \delta(q4,1) = \{q0,q1\} \cup \varnothing \cup \{q4\} = \{q0,q1,q4\}$

Portanto, como q4 pertence ao conjunto de estados finais, a seqüência 01001 pertence a L(M).

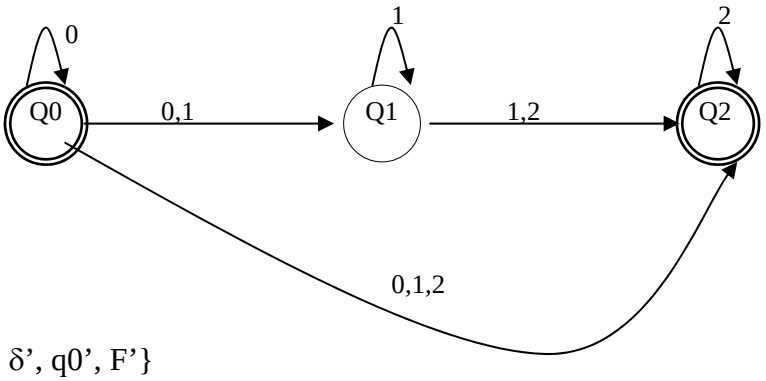
- **Equivalência entre MEFs indeterministas e MEFs deterministas *(Teorema 2.1 Passagem de Ind para Det)***

Toda MEF determinista é uma MEF indeterminista, ou seja, a classe de linguagens aceitas pelas MEFs ind inclui a linguagem regular (linguagem aceita pelas MEFs det).
As MEFs ind podem ser transformadas em MEFs det.
Na construção de uma MEF det a partir de uma MEF ind, cada estado da MEF det corresponde a um conjunto de estados da MEF ind.

Exemplo 22: aplicação do teorema 2.1.

Encontrar a MEF det equivalente a MEF ind abaixo

MEFind = {Q,Σ,δ,q0,F}
Q = {q0,q1.q2}
Σ = {0,1,2}
F = {q0,q2}



MEFdet= {Q', Σ', δ', q0', F'}
$Q' = 2^Q$
Q'= {0,q0,q1,q2, [q0,q1], [q0,q2], [q1,q2], [q0,q1,q2]

Estados	0	1	2

Q0	[q0,q1,q2]	[q1,q2]	[q2]
Q1	∅	[q1,q2]	[q2]
Q2	∅	∅	[q2]
[q0,q1]	[q0,q1,q2]	[q1,q2]	[q2]
[q0,q2]	[q0,q1,q2]	[q1,q2]	[q2]
[q1.q2]	∅	[q1,q2]	[q2]
[q0,q1,q2]	[q0,q1,q2]	[q1,q2]	[q2]

Antes de desenhar-se o diagrama de estados, elimina-se os estados que não recebem nenhuma entrada.
O estado inicial (q0) não pode ser retirado, começando por q1 verificamos se q1 recebe alguma entrada em 0,1 e 2, ou seja, olhamos para a tabela nas entradas 0,1 e 2 e verificamos se q1 aparece em algumas das entradas. No exemplo acima, q1 não aparece em nenhuma delas. Os estados [q0,q1] e [q0,q2] também podem ser eliminados, pois não aparecem em nenhuma entrada da tabela, oque significa que não recebem nenhuma informação (entrada), sendo impossível chegar até esse estado.

Para resolver em casa:

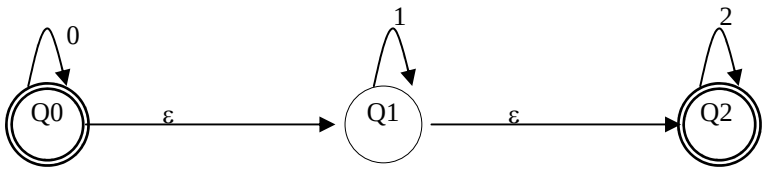
Faça o diagrama do exemplo acima, sem otimização (sem retirar os estados q1, [q0,q1] e [q0,q2]) e depois o diagrama otimizado para comprovar oque foi mencionado no texto acima.

- **MEF indeterminista com transições espontâneas – MEFind-ε**

Trata-se de uma MEF indeterminista cujo alfabeto de entrada aceita ε (entrada vazia).

Exemplo 23: Seja MEFind-ε =

Q={q0,q1,q2}
Σ = {0,1,2,ε}
F = {q2}



A tabela de transição para essa MEF é:

Estados	0	1	2	ε

Q0	Q0	∅	∅	Q1
Q1	∅	Q1	∅	Q2
Q2	∅	∅	Q2	Q2

- **Equivalência entre MEFs ind-ε e MEFind (Teorema 2.2)**

Exemplo 24: Vamos exemplificar a aplicação desse teorema na MEFind_ε dada acima. Vamos transformá-la em MEFind apenas, ou seja, retirar as entradas ε.

1. O primeiro passo dessa transformação é achar o fecho ε de cada estado, o fecho ε d um estado são todos os caminhos permitidos pela entrada ε.
   Por exemplo, o fecho ε de q0 é : {q0,q1,q2}, significando que de q0 com ε podemos chegar a q1 e desse estado com ε chegamos a q2. O fecho ε de q1= {q1,q2} e o fecho ε de q2 = {q2}. Note que sempre inclui o próprio estado.

2. O segundo passo é aplicar a função de transição dessa transformação: δ(q0,wa) = fechoε(δ(δ(q0,ε), a), encontrando assim todas transições possíveis para os estados da MEF.

Por exemplo, o fecho ε de q0 do diagrama acima é: {q0,q1,q2}
A função de transição fica assim:
δ(q0,0) = fechoε(δ(δ(q0,ε),0) = fechoε(δ(δ({q0,q1,q2},0) = fechoε(δ(δ(q0,0) U δ(q1,0) U δ(q2,0)) = fechoε({q0} U ∅ U ∅) = fechoε(q0) = {q0,q1,q2}

Para resolver em casa:
Faça as próximas transições e o diagrama otimizado dessa transformação

- **Expressões regulares**

AS linguagens aceitas por uma MEF podem ser descritas por expressões regulares. As definições das expressões regulares envolve as operações de união (R+S), concatenação (RS) e fecho (R^*). As expressões regulares relativas a um determinado alfabeto Σ, são definidas recursivamente.
∅ é uma expressão regular e representa o conjunto vazio. ε é uma expressão regular e representa {ε}. Para cada a em Σ, a é uma expressão regular e representa {a}. Se r e s são expressões regulares, representando as linguagens R e S, então (r+s), (rs) e (r^*) são expressões regulares e representam os conjuntos (R+S), (RS) e (R^*).

Seja Σ um conjunto finito de símbolos e L, l1, l2 conjuntos de seqüências de Σ^*.

- A concatenação de L1L2 = { xy | x ∈ L1 e y ∈ L2}

- O fecho $L^* = U_{i=0}^{\infty}$ Li onde L0={ε} e Li = L.L $^{i-1}$

- O fecho positivo $L^+ = U_{i=0}^{\infty}$

Exemplo 25:

Seja L1={10,1} e L2={011,11}

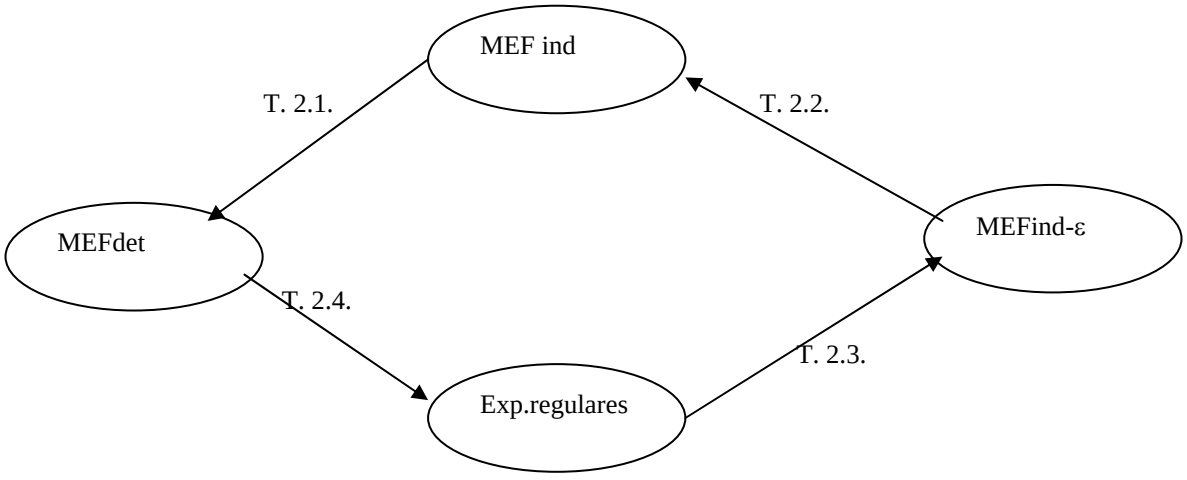
L1L2 = {10011, 1011,111}

$L1^*$ =

I=0 $L1^0 = \{\varepsilon\}$
I=1 $L1^1 = L1.L1^0 = \{10,1\}$
I=2 $L1^2 = L1.L1^1 = \{1010, 101, 110,11\}$
I=3 $L1^3 = L1.L^2 = \{101010, 10101, 10110, 1011, 11010, 1101, 1110, 111\}$

Observações:
- Precedência de operadores: fecho > concatenação > união
- rr^* pode ser abreviado para r^+

- **Equivalência entre MEFs e expressões regulares**



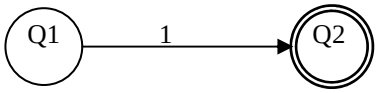
- **De expressões regulares para MEFind_ε (Teorema 2.3.)**

Exemplo 26: aplicação do teorema 2.3.:

Seja a expressão regular $01^* + 1 = (0\ (1^*) + 1$

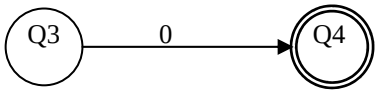
A expressão é do tipo r1 + r2 com r1=01^* e r2=1

(a) a MEF para r2 é:



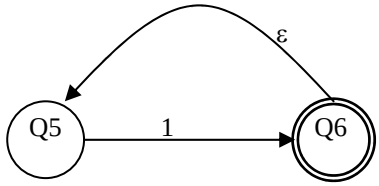
A expressão r1 é do tipo r3r4 com r3=0 e r4=1^*

(b) a MEF para r3 é:

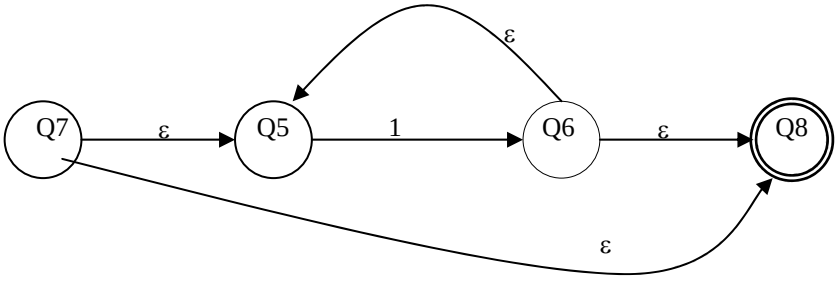


A expressão r4 é do tipo r5* com r5=1

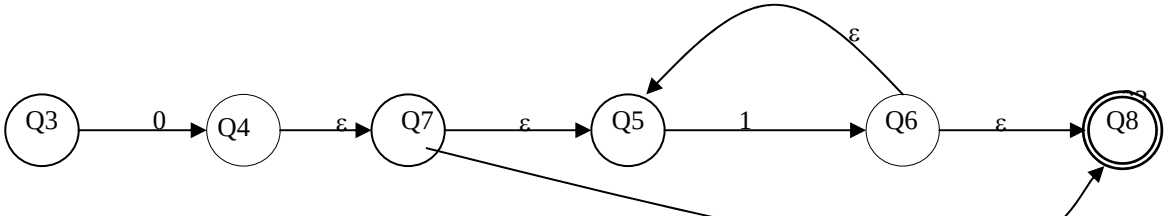
(c) a MEF para r5 é:



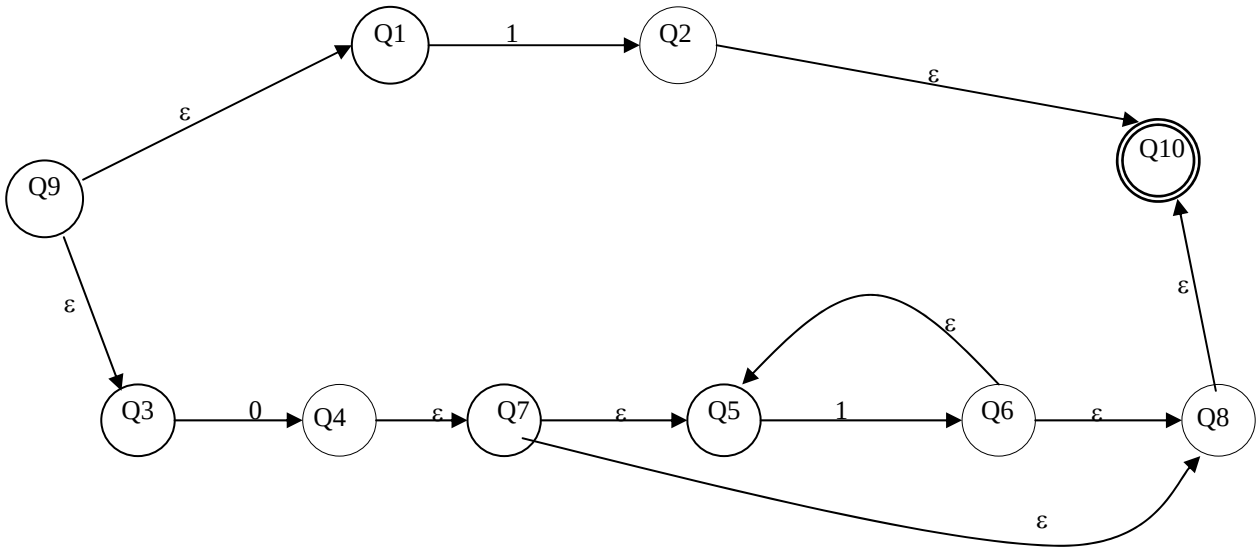
(d) a MEF para r4=r5* utiliza-se a construção para o caso (c) do teorema 2.3., introduz-se dois novos estados (q0=q7 e f0=q8)



(e) a MEF para r1=r3r4 utiliza a construção para o caso (b).



(f) a MEF para r=r1+r2 utiliza a construção para o caso (a) do teorema 2.3.Criam-se dois novos estados (q0=q9 e f0=q10)



3. ANÁLISE SINTÁTICA

Os métodos mais comumente usados nos compiladores são classificados como *top-dow* ou *bottom-up*. Como indicado por seus nomes, os analisadores sintáticos *top-dow* constróem árvores do topo (raiz) para o fundo (folhas) , enquanto que os *bottom-up* começam pelas folhas e trabalham árvore acima até a raiz. Em ambos os casos, a entrada é varrida da esquerda para direita, um símbolo de cada vez.
Os métodos de análise sintática mais eficientes, tanto *top-dow* quanto bottom-up, trabalham somente em determinadas subclasses de gramática, mas várias dessas subclasses, como as das gramáticas LL (gramáticas livres de contexto) e LR (gramáticas regulares), são suficientemente expressivas para descrever a maioria das construções sintáticas das linguagens de programação. Os analisadores implementados manualmente trabalham freqüentemente com gramáticas LL.

Os da classe mais ampla das gramáticas LR são usualmente construídos através de ferramentas automatizadas (YACC, por exemplo).
Assumimos, que a saída de um analisador sintático seja alguma representação da árvore gramatical para o fluxo de *token*s produzido pelo analisador léxico. Na prática, existe um certo número de tarefas que poderiam ser conduzidas durante a análise sintática, tais como coletar informações sobre os vários *token*s na tabela de símbolos, realizar a verificação de tipos, etc.

3.1. Tratamento dos Erros de Sintaxe

Se um compilador tivesse que processar somente programas corretos, seu projeto de implementação seria grandemente simplificado. Mas os programadores freqüentemente escrevem programas incorretos, e um bom compilador deveria ajudar o programador na identificação e localização dos erros. É gritante que, apesar dos erros serem lugar-comum, poucas linguagens sejam projetadas tendo-se em mente o tratamento de erros. Nossa civilização seria radicalmente diferente se as linguagens faladas tivessem as mesmas exigências de correção sintática que as das linguagens dos computadores.
A maioria das especificações das linguagens de programação não descreve como um compilador deveria responder aos erros, tal tarefa é deixada para o projetista do compilador. O planejamento do tratamento de erros desde o início poderia tanto simplificar a estrutura de um compilador quanto melhorar sua resposta aos erros.
Sabemos que os programas podem conter erros em muitos níveis diferentes. Por exemplo, os erros podem ser:

- Léxicos, tais como errar a grafia de um identificador, palavra-chave ou operador;
- Sintáticos, tais como uma expressão aritmética com parênteses não-balanceados;
- Semânticos, tais como um operador aplicado a um operando incompatível;
- Lógicos, tais como uma chamada infinitamente recursiva.

Freqüentemente, boa parte da detecção e recuperação de erros num compilador gira em torno da fase de análise sintática. Isto porque os erros ou são sintáticos por natureza ou são expostos quanto ao fluxo de *token*s proveniente do analisador léxico desobedecer às regras gramaticais que definem a linguagem de programação.
Outra razão está na precisão dos modernos métodos de análise sintática, podem detectar muito eficientemente s presença de erros sintáticos num programa. Detectar precisamente a presença de erros semânticos e lógicos em tempo de compilação é uma tarefa muito mais difícil. Apresentaremos umas poucas técnicas básicas para a recuperação de erros sintáticos; sua implementações são discutidas em conjunto com os métodos de análise sintática.
O tratador de erros num analisador sintático possui metas simples de serem estabelecidas:

- Deve relatar a presença de erros clara e acuradamente;
- Deve se recuperar de cada erro suficientemente rápido a fim de ser capaz de detectar erros subsequentes;
- Não deve retardar significativamente o processo de programas corretos.

Felizmente, os erros comuns são simples e freqüentemente basta somente um mecanismo de tratamento de erros relativamente direto. Em alguns casos, entretanto, um erro pode ter ocorrido muito antes de sua presença ter sido detectada e sua natureza precisa pode ser muito difícil de ser deduzida. Em casos difíceis, o tratador de erros pode ter que adivinhar o que o programador tinha em mente quando o programa foi escrito.
Vários métodos de análise sintática, tais como os métodos LL e LR, detectam os erros tão cedo quanto possível. Mais precisamente, possuem a propriedade do prefixo viável, significando que detectam que um erro ocorreu tão logo tenha examinado um prefixo da entrada que não seja o de qualquer cadeia da linguagem.

Exemplo 27: A fim de se ter uma apreciação dos tipos de erros que ocorrem na prática, vamos examinar os erros encontrados numa amostra de programa Pascal de estudantes.
Descobriu-se que os erros não ocorrem com tanta freqüência; 60 % dos programas compilados estavam semântica e sintaticamente corretos. Mesmo quando os erros ocorriam de fato, eram um tanto dispersos; 80% dos enunciados contendo erros possuíam apenas um, 13% dois. Finalmente, a maioria constituía-se de erros triviais; 90% eram erros em um único *token*.
Muitos dos erros poderiam ser classificados simplificadamente: 60% eram erros de pontuação, 20% de operadores e operandos, 15% de palavra-chave e os 5% restantes de outros tipos. O grosso dos erros de pontuação girava em torno do uso incorreto do ponto-e-vírgula.
Para alguns erros concreto, consideremos o seguinte programa Pascal.

```
(1) program prmax (input, output);
(2) var
(3)         x, y: integer;
(4) funcion max (i: integer; j: integer): integer
(5) { return maximum of integers i and j }
(6) begin
(7)         if i > j then max := i
(8)         else max := j
(9)   end;
(10) begin
(11)       readln (x, y);
(12) writeln (max (, y)
(13)  end.
```

Um erro comum de pontuação é o de usar uma vírgula em lugar de ponto-e-vírgula na lista de argumentos de uma declaração de função(por exemplo, usar uma vírgula em lugar do primeiro ponto-e-vírgula à linha (4); outro é o de omitir um ponto-e-vírgula obrigatório ao final de uma linha (por exemplo, o ponto-e-vírgula ao final da linha (4); um terceiro é o de colocar um ponto-e-vírgula estranho ao fim de uma linha antes de um else (por exemplo, colocar um ponto-e-vírgula ao fim da linha (7).

Talvez uma razão pela qual os erros de ponto-e-vírgula sejam tão comuns é que o seu uso varia grandemente de uma linguagem para outra. Em Pascal, um ponto-e-vírgula é um separador de enunciados; em PL/1 e C é um terminador.
Um exemplo típico de um erro de operador é o de omitir os dois pontos em :=. Erros de grafia em palavras-chave são usualmente raros, mas omitir o i de writeln seria m exemplo representativo.
Muitos compiladores Pascal não têm dificuldades em tratar os erros comuns de inserção, remoção e transformação. De fato, vários compiladores Pascal irão tratar corretamente o programa acima com um erro comum de pontuação ou de operador; irão emitir somente um diagnóstico de alerta, apontando a construção ilegal.
No entanto, um outro tipo de erro é muito mais difícil de se reparar corretamente. É o caso de um begin ou end ausente (por exemplo, omissão da linha (9). A maioria dos compiladores não irá tentar reparar esse tipo de erro.
Como deveria um tratador de erros reportar a presença de um erro? No mínimo, deveria informar o local no programa-fonte onde o mesmo foi detectado, uma vez que existe uma boa chance de o erro efetivo ter ocorrido uns poucos *token*s antes. Uma boa estratégia comum empregada por muitos compiladores é a de imprimir a linha ilegal comum apontando para a posição no qual o erro foi detectado. Se existir um razoável prognóstico do que o erro realmente foi, é incluída uma mensagem de diagnóstico informativa, por exemplo, “ponto-e-vírgula ausente nesta posição”.
Uma vez que o erro tenha sido detectado, como deveria o analisador sintático se recuperar? Como veremos, existe um número de estratégias gerais, mas nenhum método claramente se impõe sobre os demais. Na maioria dos casos, não é adequado para o analisador sintático encerrar logo após detectar o primeiro erro, porque o processamento da entrada restante ainda pode revelar outros. Usualmente, existe alguma forma de recuperação de erros na qual o analisador tenta restaurar a si mesmo para um estado onde o processamento da entrada possa continuar.
Uma estratégia cautelosa para o compilador é a de inibir as mensagens de erro que provenham de erros descobertos muito aproximadamente no fluxo de entrada. Em alguns casos, pode haver erros demais para o compilador continuar um processamento sensível (por exemplo, como deveria um compilador Pascal responder ao receber um programa Fortran como entrada?). Parece que uma estratégia de recuperação de erros tem que ser um compromisso cuidadosamente considerado levando em conta os tipos de erros que são mais propensos a ocorrer e razoáveis de processar.

- **Estratégias de Recuperação de Erros**

Existem muitas estratégias gerais diferentes que um analisador sintático pode empregar par se recuperar de um erro sintático. Apesar de nenhuma delas ter provado ser universalmente aceitável, uns poucos métodos têm ampla aplicabilidade. Introduzimos aqui as seguintes estratégias:

- Modalidade do desespero
- Nível de frase
- Produções de erro
- Correção global

Recuperação na Modalidade do Desespero.

Este é o método mais simples de implementar e pode ser usado pela maioria dos métodos de análise sintática. Ao descobrir um erro, o analisador sintático descarta símbolos de entrada, um de cada vez, até que seja encontrado um *token* pertencente ao conjunto designado de *tokens* de sincronização. Os *tokens* de sincronização são usualmente delimitadores, tais como o ponto-e-vírgula ou o **end**, cujo papel no programa-fonte seja claro.
Naturalmente, o projetista do compilador precisa selecionar os *tokens* de sincronização apropriados à linguagem-fonte. A correção na modalidade do desespero, que freqüentemente pula uma parte considerável da entrada sem verificá-la, procurando por erros adicionais, possui a vantagem da simplicidade e, diferentemente dos outros métodos a serem enfocados adiante, tem a garantia de não entrar num laço infinito. Nas situações em que os erros múltiplos num mesmo enunciado sejam raros, esse método pode ser razoavelmente adequado.

Recuperação de Frases:

Ao descobrir um erro, o analisador sintático pode realizar uma correção local na entrada restante. Isto é, pode substituir um prefixo da entrada remanescente por alguma cadeia que permita ao analisador seguir em frente. Correções locais típicas seriam substituir uma vírgula por um ponto-e-vírgula, remover um ponto-e-vírgula estranho ou inserir um ausente. A escolha da correção local é deixada para o projetista do compilador.
Esse tipo de substituição pode corrigir qualquer cadeia e tem sido usado em vários compiladores de correção de erros. O método foi primeiramente usado na análise sintática *top-dow*. Sua maior desvantagem está na dificuldade que tem ao lidar com situações nas quais o erro efetivo ocorreu antes do ponto de detecção.

Produções de Erros:

Se tivéssemos uma boa idéia dos erros comuns que poderiam ser encontrados, poderíamos aumentar a gramática, para uma linguagem em exame, com as produções que gerassem construções ilegais. Usamos, então, a gramática aumentada com essas produções de erro para construir um analisador sintático. Se uma produção de erro for usada pelo analisador, podemos gerar diagnósticos apropriados para indicar a construção ilegal que foi reconhecida na entradas

Correção Global:

Idealmente, gostaríamos que um compilador fizesse tão poucas mudanças quanto possível, ao processar uma cadeia de entrada ilegal. Existem algoritmos para escolher uma seqüência mínima de mudanças de forma a obter uma correção global de menor

custo. Dadas uma cadeia de entrada incorreta x e uma gramática G, esses algoritmos irão encontrar uma árvore gramatical para uma cadeia relacionada y, de tal forma que as inserções, remoções e mudanças de tokens requeridas para transformar x em y sejam tão pequenas quanto possível. Infelizmente, esses métodos são em geral muito custosos de implementar, em termos de tempo e espaço e, então, essas técnicas são correntemente apenas de interesse teórico.
Devemos assinalar que o programa correto mais próximo pode não ser aquele que o programador tinha em mente. Apesar de tudo, a noção de correção de custo mínimo fornece um padrão de longo alcance para avaliar as técnicas de recuperação de erros e tem sido usada para encontrar cadeias ótimas de substituição para a recuperação em nível de frase.

3.2. GRAMÁTICAS LIVRES DE CONTEXTO

Muitas construções de linguagem de programação possuem uma estrutura inerentemente recursiva que pode ser identificada por gramáticas livres de contexto. Um exemplo de seu uso é a definição de expressões aritméticas com balanceamento de parênteses e estruturas de bloco em linguagens de programação, como por exemplo, If-then-else.
Uma gramática livre de contexto (gramática, simplificadamente) consiste em terminais, não-terminais, um símbolo de partida e produções.

1. Os terminais são os símbolos básicos a partir dos quais as cadeias são formadas. A palavra “token” será um sinônimo de “terminal” ao falarmos a respeito de gramáticas para linguagens de programação. Por exemplo, o símbolo ‘+’ é um terminal.
2. Os não-terminais são variáveis sintáticas que denotam cadeias de caracteres.
3. Numa gramática, um não-terminal é distinguido como símbolo de partida, e o conjunto de cadeias que o mesmo denota é a linguagem definida pela gramática.
4. as produções de uma gramática especificam a forma pela qual os terminais e não-terminais podem ser combinados a fim de formar cadeias. Cada produção consiste em um não-terminal, seguido por uma seta (às vezes o símbolo ::= é usado no lugar da seta), seguido por uma cadeia de não-terminais e terminais.

Exemplo 28. A gramática com as seguintes produções define expressões aritméticas simples:

expr→ expr op expr
expr → (expr)
expr→ **id**
op → +
op → -
op→ *
op→ /
op→ ↑

Nesta gramática, os símbolos terminais são

id + - * / ↑ ( )

Os símbolos não-terminais são expr e op, e expr é o símbolo de partida.

Convenções Notacionais

A fim de evitar especificações do tipo “esses são terminais”, “esses são não-terminais” e assim por diante, iremos empregar as seguintes convenções notacionais relacionadas às gramáticas ao longo do resto deste livro.

1. Símbolos terminais:
   i) Letras minúsculas do início do alfabeto, tais como a, b, c.
   ii) Símbolos de operadores, tais como +, - etc.
   iii) Símbolos de pontuação, tais como parênteses, vírgula etc.
   iv) Os dígitos 0, 1, ...9.
   v) Cadeias em negrito como **id** ou **if**.

2. Símbolos não-terminais:
   i) Letras maiúsculas do início do alfabeto, tais como A, B, C.
   ii) A letra S, que, quando aparece, é usualmente o símbolo de partida.
   iii) Os nomes em itálico formados por letras minúsculas, como **expr** ou **cmd**.

3. As letras maiúsculas do final do alfabeto, tais como X, Y, Z, representam símbolos gramaticais , isto é, terminais ou não-terminais.
4. Letras minúsculas, ao fim do alfabeto, principalmente u, v, ..., z, representam cadeias de terminais.
5. Letras gregas minúsculas, α, β, γ, por exemplo, representam cadeias de símbolos gramaticais. Dessa forma, uma produção genérica poderia ser escrita como $A \rightarrow \alpha$, indicando que existe um único não-terminal A à esquerda da seta (o lado esquerdo da produção) e uma cadeia de símbolos gramaticais α à direita da seta (o lado direito da produção).
6. Se $A \rightarrow \alpha_1$, $A \rightarrow \alpha_2$, ... $A \rightarrow \alpha_k$ são todas produções com A à esquerda (chamamos de produções-A), podemos escrever $A \rightarrow \alpha_1 \mid \alpha_2 \mid ... \mid \alpha_k$. Chamamos α_1, α_2, ... α_k de alternativas para A.
7. A menos que seja explicitamente estabelecido, o lado esquerdo da primeira produção é o símbolo de partida.

Exemplo 29. Usando as simplificações, poderíamos escrever a gramática do exemplo anterior como:

$E \rightarrow EAE \mid (E) \mid - E$ **id**
$A \rightarrow + \mid _ \mid * \mid / \mid \uparrow$

Nossa convenções notacionais nos dizem que E e A são não-terminais, tendo E como símbolo de partida. Os símbolos restantes são terminais.

Derivações

Existem várias formas de enxergar o processo pelo qual uma gramática define uma linguagem. A visão derivacional fornece uma precisa descrição da construção *top-dow* da árvore gramatical. A idéia central aqui é que uma produção seja tratada como uma regra de rescrita, na qual o não-terminal à esquerda é substituído pela cadeia no lado direito da produção.
Por exemplo, considere a seguinte gramática para expressões aritméticas, como o não-terminal E representando uma expressão.

$$E \rightarrow E + E \mid E \mid (E) \mid - E \mid \mathbf{id}$$

A produção $\mathbf{E \rightarrow - E}$ significa que uma expressão precedida por um sinal de menos também é uma expressão. Essa produção pode ser usada para gerar expressões mais complexas a partir das mais simples, permitindo que se substitua qualquer instância de E por - E. No caso mais simples, podemos descrever essa ação escrevendo.

$$E \Rightarrow - E$$

que lido "E deriva - E ". A produção $E \rightarrow (E)$ nos diz que poderíamos também substituir uma instância de um E em qualquer cadeia de símbolos gramaticais por (E); por exemplo, $E * E \Rightarrow E * (E)$.
Podemos tomar um único E e aplicar repetidamente as produções em qualquer ordem, a fim de obtermos uma seqüência de substituições. Por exemplo,

$$E \Rightarrow - E \Rightarrow - (E) \Rightarrow (\mathbf{id})$$

Chamamos uma tal seqüência de substituições de uma derivação de - (**id**) a partir de E. Essa derivação providencia uma prova de que uma instância particular de uma expressão é a cadeia - (**id**).
Num posicionamento mais abstrato, dizemos que $\alpha A \beta \Rightarrow \alpha\gamma\beta$ se $A \rightarrow \gamma$ for uma produção e α e β forem cadeias arbitrárias de símbolos gramaticais. Se $\alpha_1 \Rightarrow \alpha_2 \Rightarrow ... \alpha_{n'}$ dizemos que α_1 deriva $\alpha_{n'}$. O símbolo $\Rightarrow$ significa "deriva em um passo". Freqüentemente desejamos dizer "deriva em zero ou mais passos". Para esses propósito usamos o símbolo $\overset{*}{\Rightarrow}$. Dessa forma,

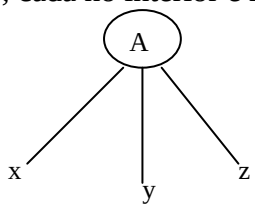
1. $\alpha \overset{*}{\Rightarrow} \alpha$ para qualquer cadeia α, e
2. Se $\alpha \overset{*}{\Rightarrow} \beta$ e $\beta \Rightarrow \gamma$, então $\alpha \overset{*}{\Rightarrow} \gamma$.

Do mesmo modo, usamos $\overset{+}{\Rightarrow}$ para significar "deveria em um ou mais passos".
Dada uma gramática G, com símbolo de partida S, podemos usar a relação $\overset{+}{\Rightarrow}$ para definir L (G), a linguagem gerada por G.

Resumindo, uma gramática deriva cadeias começando pelo símbolo de partida e, então, substituindo repetidamente um não-terminal. As cadeias de tokens que podem ser derivadas a partir do símbolo de partida formam a linguagem definida pela gramática.

- **Árvores Gramaticais**

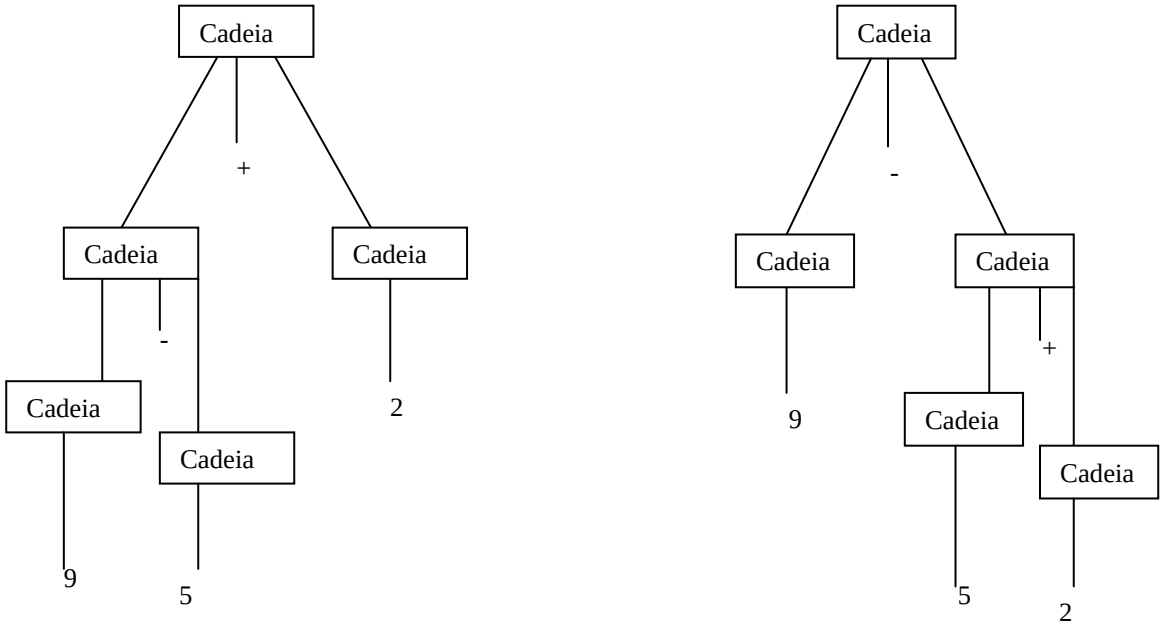
Uma árvore gramatical mostra como o símbolo de partida de uma gramática deriva uma cadeia da linguagem. Se um não-terminal A possui uma produção A⇒ xyz, então uma árvore gramatical pode ter um nó interior rotulado A, com tr6es filhos rotulados x,y e z. Ou seja, a raiz é rotulada pelo símbolo de partida da gramática, cada folha é rotulada por um token (terminal) ou por ε, cada nó interior é rotulado por um não terminal.



As folhas da árvore gramatical, lidas da esquerda para a direita, formam o produto da árvore, que é a cadeia gerada ou derivada a partir do não terminal na raiz da árvore gramatical. O processo de encontrar uma árvore gramatical para uma dada cadeia de tokens é chamado de análise sintática daquela cadeia.

- **Ambigüidade**

Uma gramática pode ter mais que uma árvore gramatical gerando uma dada cadeia de tokens. Tal gramática é dita ambígua. Por exemplo, duas árvores gramaticais para 9-5+2



Como uma cadeia com mais de uma árvore usualmente possui mais de um significado, precisamos, para as aplicações de compilação, de gramáticas não-ambíguas.

- **Associatividade dos Operadores**

Por convenção, 9+5+2 é equivalente a (9+5)+2. Quando um operando, como 5, possui operadores à esquerda e à direita, são necessárias convenções para decidir que operador recebe qual operando.
Dizemos que o operador + associa-se à esquerda, porque um operando com sinais de adição em ambos os lados é absorvido pelo operador à sua esquerda. Na maioria das linguagens de programação, os quatro operadores matemáticos (+, - , *, \) são associativos à esquerda.
Alguns operadores comuns, tais como exponenciação e a atribuição são associativos à direita. Cadeias como a=b=c, com um operador associativo à direita são geradas pela seguinte gramática.

direita⇒letra = direita | letra
letra⇒a|b|...|z

- **Precedência de Operadores**

Considere a expressão 9+5*2, existem duas possíveis interpretações para a mesma expressão: (9+5)*2 ou 9+(5*2). A associatividade do operador + e do operador * não resolvem esta ambigüidade. Para resolver determinadas ambigüidades necessitamos de dois conceitos: associatividade e precedência de operadores.
Uma gramática para expressões aritméticas pode ser construída a partir de uma tabela mostrando a associatividade e a precedência de operadores.
Por exemplo, começamos pela tabela

Associatividade à esquerda	+ -	
Associatividade à direita	* \	em relação à precedência

Em seguida, criamos dois não terminais, Expressão e Termo, para os dois níveis de precedência, e um não terminal chamado Fator para gerar as unidades básicas das expressões que são dígitos ou expressões com parênteses.

fator⇒digito |(expressão)

Consideremos agora os operadores binários que possuem a preced6encia mais alta.

termo⇒termo*fator | termo\fator | fator

Enfim, definimos o símbolo inicial expressão.

expressão⇒ expressão + termo | expressão – termo | termo

A gramática resultante é:

expressão⇒ expressão + termo | expressão – termo | termo
termo⇒termo*fator | termo\fator | fator
fator⇒digito |(expressão)

- **Restrições na prática de gramáticas**

Uma gramática que gere regras inúteis (que nunca são usadas) pode gerar erros ou tornar-se ambígua. O reconhecimento de sentenças é mais fácil se a gramática não conter regras desnecessárias.
Um símbolo não terminal deve satisfazer duas condições para poder aparecer na derivação de uma sentença:

1. <u> deve aparecer de forma sentencial , <u>⇒x<u>y
2. deve derivar uma cadeia de símbolos não terminais a partir do mesmo.

Resumindo, alguma string terminal ou não terminal deve derivar de <u> e <u> deve ocorrer em alguma string derivada do símbolo inicial da gramática.

Exercícios de gramática no caderno
Obs: resolução dos exercícios será feita em sala de aula com a participação dos alunos

3.3. Projeto de Analisador Sintático

O trabalho a ser realizado nessa disciplina tem por finalidade construir as duas primeiras fases de um compilador de linguagem de programação, um mini-Pascal. O compilador poderá ser feito em Pascal, C, Delphi, já que a parte mais importante é a definição do projeto.
Inicialmente, devemos ter em mente quais estruturas e dados o compilador deve aceitar. Por ser este um trabalho acadêmico, o nosso Pascal ficou extremamente reduzido, com as seguintes especificações:

Tipos:

Integer (signed 16-bit [-32768 .. 32767])
Boolean (1 byte [True, False])

Constantes:

Numéricas (Integer)
Boolean (True, False)
Strings

Entrada/Saída:

Read, ReadLn
Write, WriteLn

Laço:

While .. do

Condicional:

If .. Then .. Else

Operadores Aritméticos:

+, -, *, /

Operadores Relacionais:

=,<> <, <=, >, >=
Comentários:

{ ... }
Atribuições:

:=

Strings:

'...'

A linguagem, consequentemente, não aceitará construções repeat .. until, goto, case .. of, procedimentos, funções, vetores, array, record, nem outros tipos de dados.

Para implementarmos uma gramática que reconheça essas especificações, utilizamos duas ferramentas: EBNF (Extended Backus-Naur Form) e Notação Formal, esta última usando GLC (Gramática Livre de Contexto).

- **Elementos Básicos e Especificações de EBNF**

Conjunto de Símbolos Terminais: são os símbolos que formam a linguagem. Ex.: begin, while, etc.;

Conjunto de Símbolos Não-Terminais: são os símbolos que não pertencem à linguagem, são usados para representar definições intermediárias usadas nas produções da gramática. São cercados por '<' e '>';
Conjunto de Produções: é a definição de um símbolo não-terminal.

Forma Geral:

X::=Y

onde
X - símbolo não-terminal
Y - seqüência de símbolos

Símbolo Alvo (ou Inicial): não-terminal especial.

Obs.:
1. Todo símbolo não-terminal deve aparecer na esquerda de alguma produção.
2. O símbolo alvo não pode aparecer na direita de produção.

Podemos representar ainda:

Alternativa: |
Opcional: [ ]
Repetição: { }

Obedecendo essas regras, nossa gramática implementada em EBNF teria 14 não-terminais e pode ser vista na tabela abaixo.
Obedecendo essas regras, nossa gramática implementada em EBNF teria 14 não-terminais e pode ser vista na tabela abaixo.

<programa>::=program id; [<declarações>] begin <lista comandos> end.

<declarações>::=var id {,id} : <tipo>; {id {,id} : <tipo>;}

<tipo>::=integer | boolean | real | char

<lista comandos>::=<comando> ; {<comando>;}

<comando>::=<atribuição> | <leitura> | <escrita> | <composto> | <condicional> | <repetição>

<atribuição>::=id := <expr>

<leitura>::=read (id {,id}) | readln [(id {,id})]

<escrita>::=write (<stringvar> {,<stringvar>}) | writeln [(<stringvar> {,<stringvar>})]

<composto>::=begin <lista comandos> end

<condicional>::=if <exprboolean> then <listacomandos> [else <listacomandos>]

<repetição>::=while <exprboolean> do <listacomandos>

<expressão>::= <expressão> + <termo> | <expressão> – <termo> | <termo>

<termo>::=<termo>*<fator> | <termo> \ <fator> | <fator>

<fator>::=<num> | id | (<expressão>)

<exprboolean>::=<expr> < <expr> | <expr> <= <expr> | <expr> > <expr> | <expr> >= <expr> | <expr> = <expr> | <expr> <> <expr> | id

<stringvar>::=str | <expressão>

<id>::= letra { letra | digito}

<num>::= inteiro | fracionario | exponencial

<letra>::= a .. z, A .. Z

<digito>::= 0..9

- **< Elementos Básicos e Especificações de Notação Formal**

Uma gramática G é definida pela quádrupla:

G = (N,T,P,S)

onde:
N = conjunto finito de não-terminais;
T = conjunto finito de terminais;
P = conjunto finito de regras de produção;
S = símbolo inicial.

Especificação de GLC

Para G sendo a quádrupla (N,T,P,S) e V+ sendo N unido com T exceto o vazio temos:

P = {a->b | a pertence a N e b pertence a V+}

Obedecendo essas especificações, construímos uma gramática em notação formal, que pode ser vista na tabela abaixo. Foram utilizados 17 não-terminais.

S->program id; D begin L end. | program id; begin L end.
D->var V
V->I: T;V | I : T;
T->integer | boolean
I->id | id,I
L->C; | C;L
C->A | R | W | M | N | P
A->id := E
R->read (I) | readln | readln (I)
W->write (F) | writeln | writeln (F)
F ->G | G,F
G->str | E
M->begin L end
N->if B then C | if B then C else C
P->while B do C E->E + E | E - E | E * E | E / E | - E | (E) | id | num
B->E < E | E <= E | E > E | E >= E | E = E | E <> E | id

Com relação a EBNF anteriormente mostrada, podemos fazer a seguinte ligação
EBNF->GLC:

S=<programa> W=<escrita> D=<declarações> F=lista de <stringvar> V=lista de variáveis na declaração G=<stringvar> T=<tipo> M=<composto> I=lista de ids N=<condicional> L=<lista comandos> P=<repetição> C=<comando> E=<expr> A=<atribuição> B=<exprboolean> R=<leitura>

Assim, as definições gramaticais para a realização de nosso trabalho já foram feitas, exigindo de vocês apenas a compreensão dessas regras, sem a qual não será possível programá-las.
A fim de facilitar a construção desse analisador apresento um método simples para construção de analisadores sintáticos: Análise descendente recursiva.

3.3.1. Análise Sintática Descendente Recursiva

É um método que emprega uma subrotina para cada não-terminal da gramática. Elas chamam-se entre si, havendo recursão entre elas. Uma gramática para esse tipo de analisador não pode ser ambígua, não deve ter recursão à esquerda e deve estar fatorada à esquerda. Por exemplo, a gramática

Expr -> Expr + Termo | Expr - Termo | Termo
Termo -> Termo * Num | Termo / Num | Num
Num -> 0..9

não é adequada para esse tipo de analisador, pois não está fatorada à esquerda e possui recursão à esquerda. Iremos adaptá-la, em primeiro lugar eliminaremos a recursão à esquerda de Expr e Termo.
Vejamos, se uma gramática possui produções como A -> Aα1 | β iremos transformá-la em A -> βA`

A`-> α1A` | ε

assim temos que: Expr -> Expr + Termo | Expr - Termo | Termo pode ser transformada em:
Expr -> Termo Expr`
Expr` -> + Termo Expr` | - Termo Expr` | ε

Fazendo-se o mesmo com Termo temos:
Termo -> Num Termo`
Termo` -> * Num Termo` | / Num Termo` | ε

A recursividade de um não-terminal como Expr` é transformada em iteração pelo uso de { } que indica zero ou mais vezes. Assim, A -> a{b} gera a, abb, ab, ...
Aplicando essa notação obteremos para a gramática anterior: Expr -> Termo{(=|-)Termo}

Termo -> Num{(*|/)Num}

Para cada não-terminal da gramática, existe uma função que o analisa.

- **Analisadores Preditivos**

Para construir-se um analisador preditivo precisamos conhecer, dado o símbolo de entrada **a** e o não-terminal **A**, qual das produções de **A** é a única que deriva uma cadeia começando por **a.** Por exemplo,

Cmd -> **if** expr **then** cmd **else** cmd | **while** expr **do** cmd | **begin** listacmds **end**

As palavras-chave if, while e begin nos informam qual é a única alternativa possível. Ou seja, o símbolo corrente da entrada determina a seleção da produção a ser usada.
O analisador preditivo do exemplo abaixo consiste em procedimentos para os não-terminais da gramática e um procedimento adicional (reconhecer()), usamos reconhecer() a fim de simplificar o código. Reconhecer avança para o próximo token da entrada se seu argumento T for igual ao símbolo corrente.
Exemplo 30: Consideremos a gramática abaixo

tipo -> tipo_simples | ↑ id | array[tipo_simples] of tipo
tipo_simples -> integer | char | num pontoponto num

A análise gramatical começa com uma chamada ao procedimento correspondente no não-terminal de partida de nossa gramática tipo.

```
Procedimento reconhecer(t:token)
Inicio
        Se tokencorrente = t então
                tokencorrente := prox_token
        Senão erro
Fim;

Procedimento tipo;
Inicio
        Se tokencorrente está em {integer, char, num} então tipo_simples
        Senão se tokencorrente = "↑" então
                                Inicio
                                    Reconhecer("↑"); reconhecer(id)
                                Fim
        Senão se tokencorrente = array então
                               Inicio
                                   Reconhecer(array); reconhecer("["); tipo_simples;
                                   Reconhecer("]"); reconhecer(of); TIPO
                               Fim
        Senão erro
Fim;

Procedimento tipo_simples;
Inicio
        Se tokencorrente = integer então reconhecer(integer)
        Senão se tokencorrente = char então reconhecer(char)
        Senão se tokencorrente = num então
                                        Inicio
                                          Reconhecer(num); reconhecer(pontoponto);
                                          Reconhecer(num)
                                        Fim
        Senão erro
Fim;
```

3.4. Especificações Lex

Um programa Lex é constituído de 3 partes:

```
Declarações
%%
regras de tradução
%%
```

procedimentos auxiliares

A seção de declarações inclui declarações de variáveis, constantes e definições regulares. As regras de tradução de um programa Lex são enunciadas como,

P1 {ação1}
. . .
Pn {açãon}

Onde cada Pi é uma expressão regular e cada açãoi é um fragmento de programa descrevendo que ação o analisador léxico deverá tomar quando o padrão pi reconhecer um lexema. Em Lex, as ações são escritas em C.
A seção de procedimentos contém qualquer procedimento auxiliar que seja necessário para as ações. Esses procedimentos podem ser compilados separadamente e carregados com o analisador léxico.
Um analisador léxico gerado pelo Lex se comporta em relação ao parser da seguinte forma:

-quando ativado pelo parser, começa lendo suas entradas, um caracter de cada vez, até encontrar uma entrada que seja reconhecida por uma das expressões regulares pi, em seguida, executa açãoi.

Tipicamente, o analisador Lex retorna uma única quantidade, o token, para o parser. A fim de passar um valor de atributo, com informações sobre o lexema, podemos atribuir valores a variável global denominada **yylval**.

Exemplo 31:

```
% { /* definições das constantes LT, LE, EQ, NE, GT, IF, THEN, ELSE, ID, NUM, RELOP */
%}
/*definições regulares*/
delim       [\t\n]
ws          {delim}+
letra       [A-Za-z]
digito      [0-9]
id          {letra} ({letra} |  {digito})*
num         {digito} + (\. {digito}+)? (E[+|-]?{digito}+)?
%%
{ws}    {/*nenhuma ação*/}
if      {return(IF);}
then    {return(THEN);}
else    {return(ELSE);}
{id}    {yylval=instalar_id(); return(ID);}
{num} {yylval=instalar_num(); return(NUM);}
"<"     {yylval=LT; return(RELOP);}
. . .
```

```
%%
instalar_id() {/* procedimento para instalar o lexema na tabela de símbolos e retornar um
apontador para o mesmo*/}
. . .
}
```

Na seção de declarações vemos a declaração de constantes usadas pelas regras de tradução, elas são envolvidas por chaves especiais %{ e %}. Qualquer coisa figurando entre estas chaves é copiada diretamente no analisador léxico **lex.yy.c** e não é tratada como parte das definições regulares ou das regras de tradução. Também incluídas na seção de definição estão algumas definições regulares, cada definição consiste em um nome e uma expressão regular denotada por aquele nome.
Ex: delim figura no lugar da classe de caracteres [\t\n]
Na última definição regular, a de número, vemos ? significando "zero ou uma ocorrência de".

- **Gerador De Analisadores Sintáticos Yacc**

Em primeiro lugar, um arquivo nome.y contendo uma especificação yacc é preparado, o comando yacc nome.y transforma o arquivo em um programa C chamado **y.tab.c**.
Através da compilação de y.tab.c obtemos o programa-objeto **a.out** que realiza a tradução especificada pelo programa yacc. Se outros procedimentos forem necessários podem ser compilados e carregados com y.tab.c exatamente como em qualquer outro programa C.
Um programa-fonte Yacc possui 3 partes,

```
Declarações
%%
regras de tradução
%%
rotinas de suporte C
```

Exemplo 32: Calculadora de mesa simples

Gramática: E-> E+T | T digito-> [0-9]
T-> T*F | F
F-> (E) | digito

```
%{
#include <ctype.h>
%}
%token          DIGITO
%%
linha   : expr '\n' {printf("%d\n",$1;}
        ;
expr    : expr '+' termo {$$=$1+$3;} | termo
        :
```

```
termo   : termo '*' fator {$$=$1*$3;} | fator
        ;
fator   : '(' expr ')' {$$=$2;} | digito
        ;
%%
yylex() {
        int c;
        c=getchar();
        if (isdigit(c)) {
                yylval=c - '0';
                return DIGITO; }
        return c;
        }
```

Existem duas seções opcionais na parte de declarações de um programa yacc. Na primeira colocamos declarações C delimitadas por %{ e %}. Na Segunda estão os tokens da gramática.

Na parte das regras de tradução após o primeiro par %%, cada regra consiste numa produção gramatical e uma ação semântica associada. Uma ação semântica Yacc é uma seqüência de enunciados em C, o símbolo $$ se refere ao valor de atributo associado ao não-terminal á esquerda. Enquanto $i se refere ao valor associado ao iésimo símbolo gramatical á direita.

Um conjunto de produções que na gramática representamos por: <ladoesq> -> <p1> | <p2> | <p3> seria escrito em yacc como:

```
<ladoesq>  :  <p1>  {ação semântica1}
      | <p2> {ação semântica2}
              | <p3> {ação semântica3}
      ;
```

Se o Lex for usado para produzir o analisador léxico iremos substituir a rotina yylex() na terceira parte, pelo enunciado #include "lex.yy.c".